CODERDOJO BELGIUM · REGISTRATION PLATFORM

Technical foundation and data model

How the site is built, how its data fits together, and why it looks the way it does. A summary of `DATA_MODEL.md` and `CLAUDE.md`, with the diagrams taken from the former.

Contents

1. What the site is
2. Technical foundation
3. Architectural principles and their rationale
4. The data model at a glance
5. Accounts, families and roles
6. Dojos, teams and access
7. Sessions, registrations and attendance
8. Badges and belts
9. Volunteers: applications and background checks
10. Pathways, content, notifications and geography
11. The mail engine
12. Security
13. Privacy by design
14. Languages, testing and operations
15. Capacity, load and disk
16. Coding standards and quality

Generated from the repository, 3 October 2026 · user-journeys/scripts/build_technical.py

1 What the site is

A Django site for CoderDojo Belgium: free coding clubs for children aged 7–17, run by volunteers. It replaces spreadsheets and forms with one place where families find a dojo and sign their children up, volunteers are vetted once and help at any dojo, dojo teams run their sessions, and the organisation communicates, promotes and keeps the whole thing lawful.

Who	What they do on the site
Visitors and families	Discover dojos and sessions, create a family account, sign children up (with a waiting list), follow belts and badges, manage mail, security and their data.
Ninjas	Optionally their own login, given by a guardian: see their progress, sign themselves up.
Mentors	Apply once, pass a Belgian background check, then join any dojo's team: sessions, attendance, belts and badges, the team.
Champions	Run a dojo: everything a mentor does plus its lifecycle, health notes, mail to the families and API clients.
The organisation	Reviewers (background checks, applications), admins (campaigns, segments, content, awards, privacy, people, security), the board; the Django admin only for technical fixes.

Where the knowledge lives

- `DATA_MODEL.md`: the data model, one Mermaid diagram per area, plus the design rationale and plan of every larger change (sections 10–25). The diagrams in this document are taken from it unchanged.
- `CLAUDE.md`: the conventions, the architecture and the gotchas: what a developer (human or AI) needs to change the code safely. Kept current in the same change as the code.
- `docs/`: the end-user help centre (Sphinx, English, French and Dutch).
- `user-journeys/`: one PDF per persona with screenshots, in English, Dutch and French.
- `CAPACITY.md`, `MONITORING.md`, `CODING_STANDARDS.md`, `MEMORY_PROFILE.md` and `MAINTENANCE.md`: what was measured (load, growth, memory, every metric and dashboard panel), the standards and their tooling, and the versions in use with their end-of-life dates and the security log.

In numbers

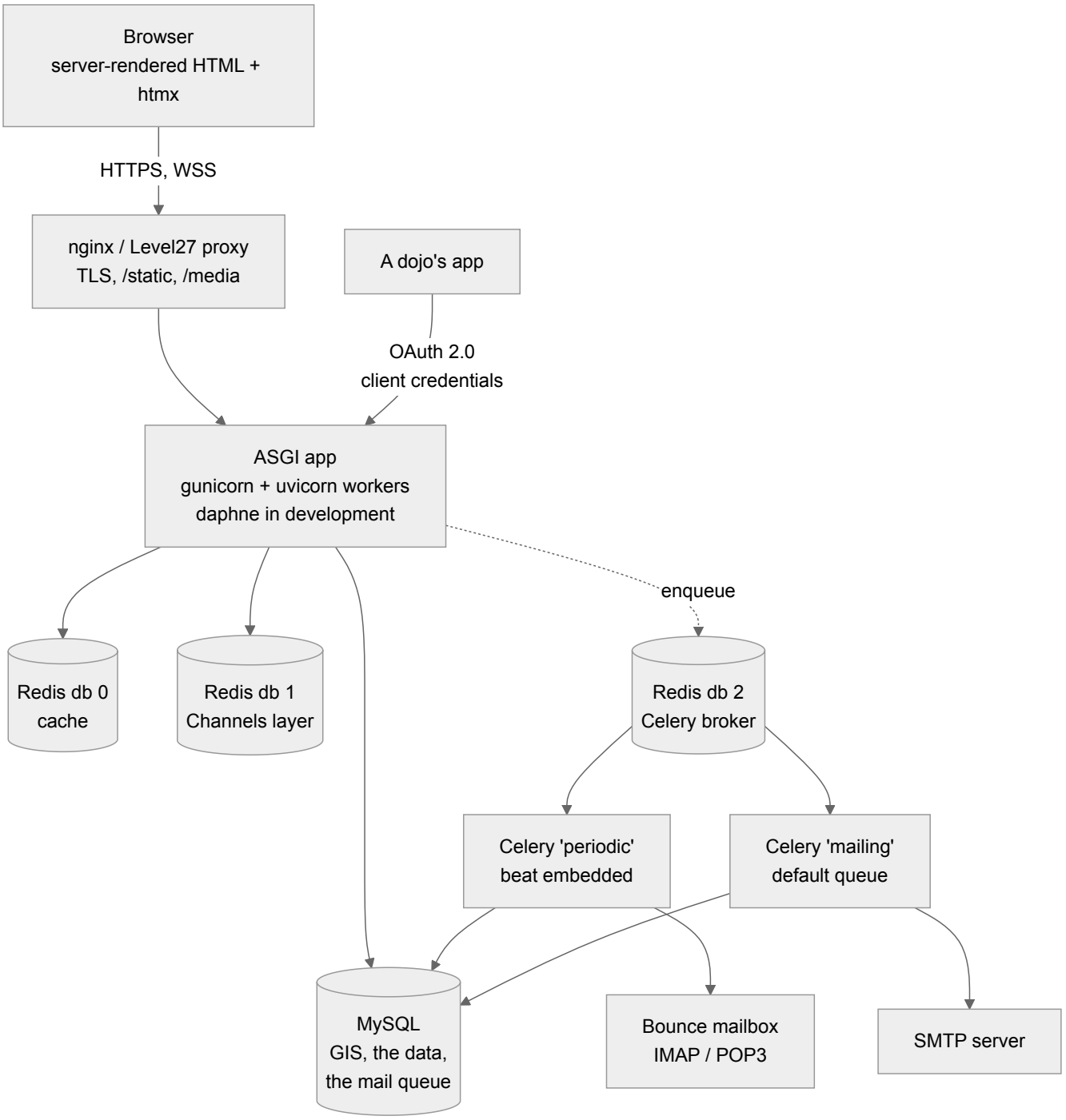
Django apps	15 of our own (plus `website` for settings)	**Models**	52 of our own
Test functions	about 1,150 (Django test runner)	**Languages**	English, Dutch, French

2 Technical foundation

The stack

Layer	Choice	Why
Language, framework	Python 3.14, Django 6.1	Batteries included (auth, admin, forms, i18n, migrations); the admin doubles as the always-working emergency tool.
Database	MySQL 8.4 LTS with `django.contrib.gis`	What the hosting offers; spatial columns for dojo locations, postcodes and provinces.
Front end	Server-rendered templates + htmx 2.0 (served from our own static files), one hand-kept `bundle.js` / `bundle.css`	No build step, no SPA: the server stays the single source of truth; htmx swaps fragments for the dynamic parts.
Serving	ASGI: gunicorn with uvicorn workers in production, daphne behind `runserver` in development	WebSockets for the live notification bell, in the same process as the pages.
Real time	Django Channels over Redis (db 1)	The bell updates itself without polling.
Background work	Celery 5.6, two workers, beat embedded in one; Redis db 2 as broker	Every mail, the nightly engagement rebuild, retention and reminders run off the request.
Cache	Redis db 0 (`IGNORE_EXCEPTIONS`)	Public lists; the site keeps working if Redis hiccups.
Auth	django-two-factor-auth on django-otp, WebAuthn passkeys; login links	Optional two-step login for everyone, enforceable per role by the organisation.
API	django-ninja + django-oauth-toolkit (client credentials)	Typed endpoints with an OpenAPI spec; apps act for one dojo with scoped tokens.
Audit	django-auditlog	Who changed what, and who viewed the most sensitive data.
Hosting	Level27 Python hosting (SSH, systemd user units); a devcontainer for development	A small, affordable setup for a volunteer organisation, deployed with one script.

How the pieces connect



Runtime architecture. Only nginx is exposed; in development it also serves the docs, Mailpit, phpMyAdmin and a test authenticator under their own paths.

Environments

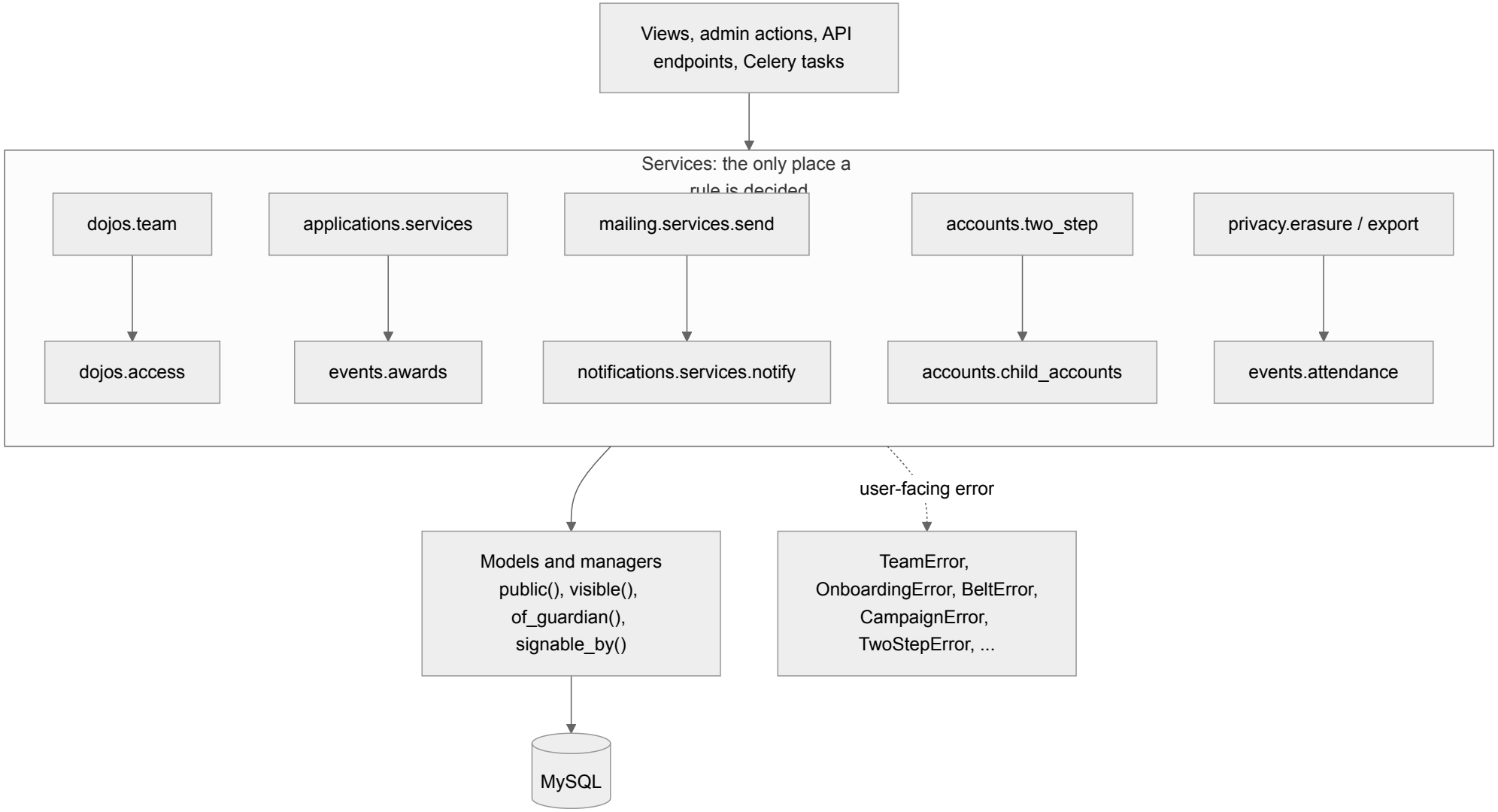
- **Development** is the `.devcontainer` stack: nginx with a local CA for `coolregistration.localhost` (only port 443 is published), MySQL, Redis, Mailpit catching every mail (and acting as the bounce mailbox), both Celery workers, and seeded demo data from about twenty rerun-safe seed commands. Prometheus and Grafana (scraping `/metrics/`, with a provisioned dashboard) come up behind a Compose profile when wanted. End-to-end checks go through nginx, never straight to Django, because TLS, the WebSocket upgrade and the tool paths live there.

- **Production** is deployed by `scripts/deploy.sh`: bundle the tree, install requirements into the Python env gunicorn uses, run `manage.py check` on the new code *before* touching the live app, rsync, migrate, reload gunicorn, restart the Celery units and smoke-test. `--check` is a read-only preflight.
- **CI** (GitHub Actions) runs the checks, never a deploy: the whole test suite on every push and pull request against MySQL 8.4 and Redis 7.2, a weekly and per-push code audit (`ruff` with its security rules, the import layers, `pip-audit`, `check --deploy`, CodeQL), and the help centre published to GitHub Pages.

3 Architectural principles and their rationale

A handful of rules shape almost every part of the code. Each one exists because the alternative went wrong, or would have, for a site that holds children's data and is maintained by volunteers.

Rules live in services, views only call them



Every change with a rule behind it goes through one module, which raises an error with a user-facing message.

> **Why.** One place decides each rule, so a view, an admin action, the API and a Celery job can never disagree. The API's attendance endpoints call the same `events.attendance` functions as the dojo's attendance list.

404, not 403, for what you may not see

A dojo you are not on the team of, a child you are not a guardian of, an organisation page your role doesn't open: all answer 404. A 403 is only for someone who *is* on the team but whose role lacks the capability.

> **Why.** A 403 confirms the thing exists. Guessable ids must not reveal which dojos, children or accounts are there.

Keep the history, never overwrite it

- Team memberships go `dormant`, never deleted, so past sessions keep their team.
- Belts (`NinjaBelt`), background-check decisions, consent (`ConsentEvent`) and cancellations (`RegistrationCancellation`) are append-only logs.
- A campaign freezes its segment at launch; a mail row stores exactly what was sent.

> **Why.** Insurance, disputes and GDPR accountability all need to know what was true *then*, not what is true now.

Delete what you must not keep

A criminal-record extract is deleted the moment a reviewer decides; only the decision stays. Private files have no URL at all (`private_storage`, `base_url=None`), only a permission-checked download view.

The Django admin always works

Day-to-day work gets a page in a dashboard (the dojo area or the organisation area). The admin stays fully usable for every model, never read-only, never with hidden fields, so that direct database access is never needed; a test checks that a superuser can add, change and delete every registered model. The one exception is the audit log. Access to the admin itself is time-boxed: an organisation role asks for it, 12 hours at a time.

Server-driven UI

htmx for anything that needs the server, plain HTML/CSS for pure client behaviour, hand-written JavaScript only for what neither can do (passkeys, geolocation, scroll). Third-party browser code (htmx, OpenLayers, fonts) is served from our own static files, never a CDN, so visitors' IP addresses don't go to a third party.

Correctness traps turned into rules

- **Distances** always use `geo.functions.DistanceSphere` (MySQL `ST_Distance_Sphere`): Django's `Distance()` silently returns planar degrees on MySQL.
- **A model's `__str__` never queries**: under ASGI a lazy lookup raises `SynchronousOnlyOperation`; managers preload what `__str__` needs.

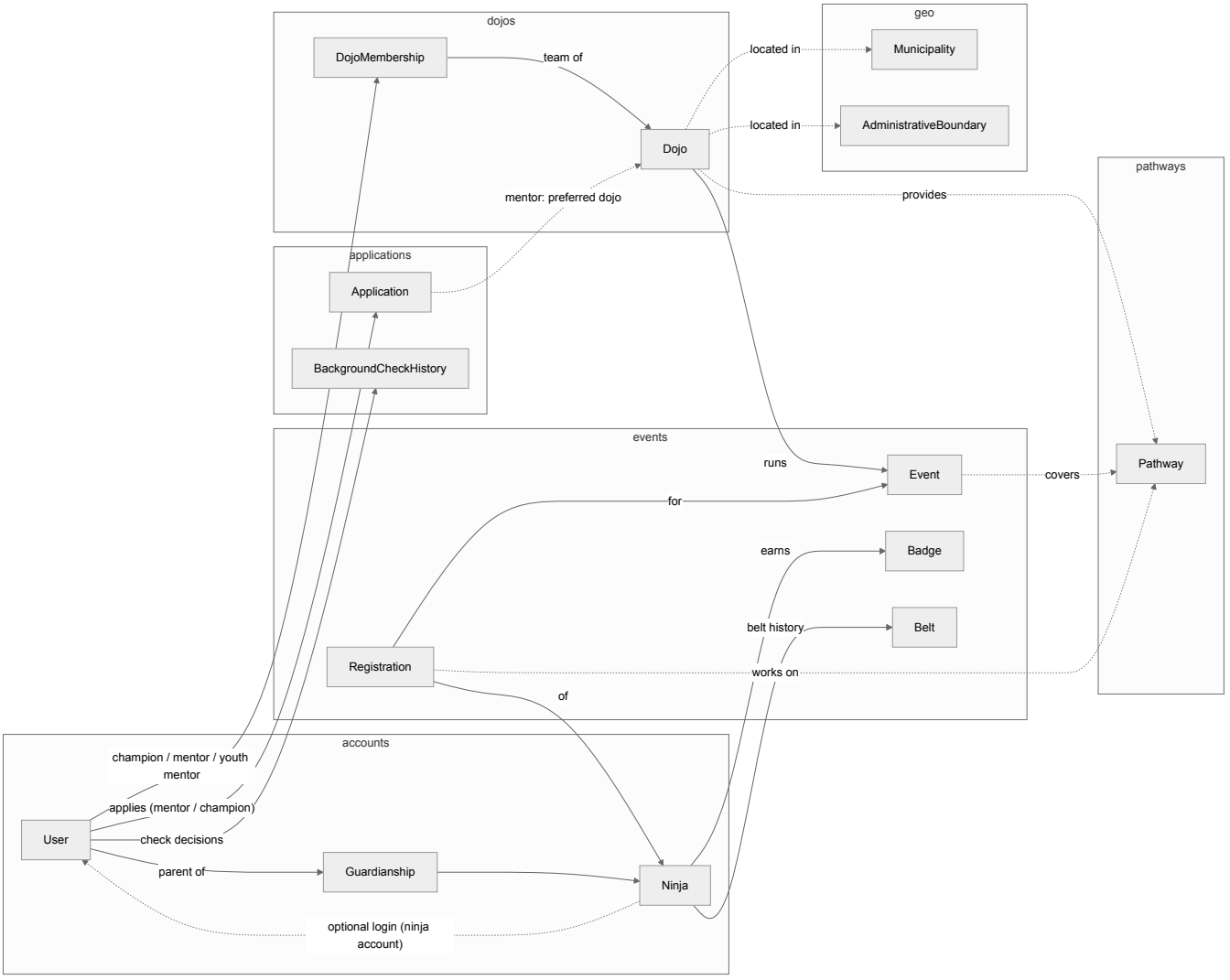
- **Cache data, never whole pages, and clear it on save**: every page carries its own CSP nonce, CSRF token and nav, so the public content, the account's nav and the sessions are cached instead (`core/caching.py`), and signals clear them whenever a row they're built from changes, so a published session shows at once. The account's nav never decides access: the pages behind its links still ask the database.
- **A list counts in the same query**: `Event.objects.with_confirmed_count()` wherever the places left show, instead of a query per row.
- **Tasks are safe to run twice** (`acks_late`, idempotency keys): a worker that dies mid-task is retried.

Enforced by tests

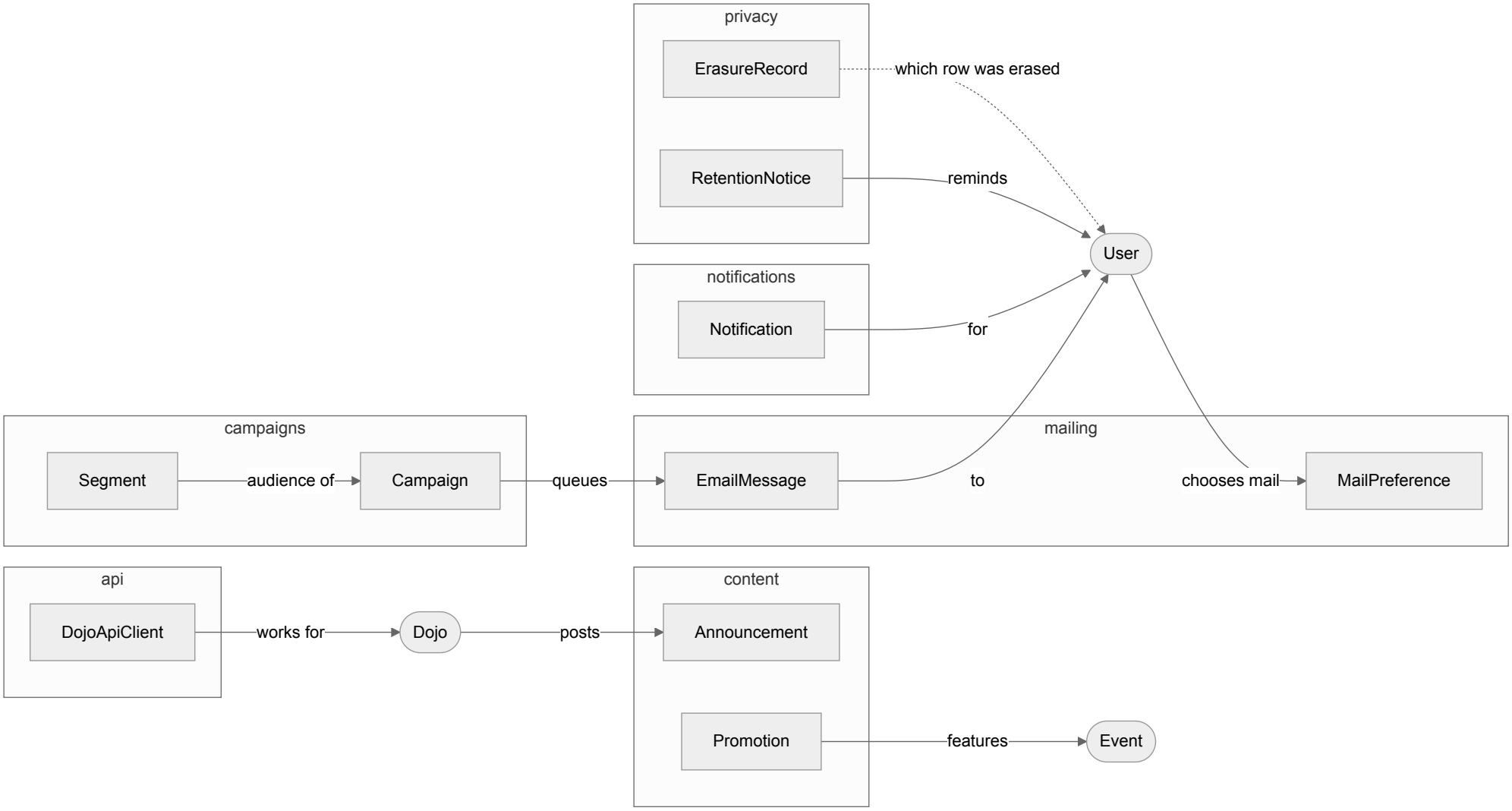
Many of these are guarded by tests that fail on a new model or field that forgets them: every field has a privacy classification, every model has an audit-log decision, every registered model is fully usable in the admin, every `__str__` that follows a relation is preloaded, every form text is translated, the API schema never exposes a sensitive field, no page loads a script from another site, and the public pages keep their query counts.

4 The data model at a glance

Fifteen apps, in four layers, each with its own `urls.py` mounted at the top level. The diagrams show the main entities and how they connect; the layers are below.



The main entities, grouped by the app that owns them (DATA_MODEL.md §1).



Around the core: the apps that reach people and keep the record (DATA_MODEL.md §1).

App	Owns
`accounts`	`User` (every login), `Ninja`, `Guardianship`, organisation roles, admin-access grants, invitations, the sign-in policy
`dojos`	`Dojo` (with its lifecycle), `DojoMembership` (the team), access rules, geo-search
`events`	`Event`, `Registration`, cancellations, badges, belts, team attendance, the engagement snapshot
`applications`	`Application`, `BackgroundCheckHistory`, the background-check flow
`pathways`	The learning-track catalogue: pathways, steps, projects, skills
`content`	FAQs, testimonials, announcements, promotions, sponsors, the organisation's team listing
`notifications`	Per-recipient `Notification` rows behind the bells
`mailing`	The mail engine: every mail's queue, templates, preferences and consent, mutes, bounces, the automated mail
`campaigns`	Campaigns, segments, journeys and a dojo's own mail, on top of the engine
`geo`	Municipalities and administrative boundaries, geocoding, `DistanceSphere`
`privacy`	The GDPR operations: the register, export, erasure, retention, erasure records
`api`	A dojo's API clients and the `/api/v1/` endpoints
`core`	The foundation: caching, uploads, the image library, the privacy registry, audit-log wiring, shared templates and static files
`pages`	The homepage, contact, `/manage/`'s landing, the audit log page, `/health/`, the Django admin site
`monitoring`	The site's measurements of itself: `/metrics/`, the daily capacity sample

Four layers

Layer	Apps
Top: the site as people meet it	`pages`
Across the domain	`api`, `campaigns`, `privacy`
The domain, with the mail engine and notifications	`accounts`, `applications`, `content`, `dojos`, `events`, `mailing`, `notifications`, `pathways`
Foundation: knows no app	`core`, `geo`, `monitoring`

An app imports its own layer and the ones below, never one above; `lint-imports` (import-linter) checks it in CI. When a lower layer needs something from a higher one, the higher one registers it at startup: the standard-image folders, `/metrics/`'s mail queue, which campaigns use a mail template.

> **Why.** Before the split, core imported most apps and 24 pairs of apps imported each other, so any change could ripple anywhere. With the layers, the foundation and the mail engine can be changed without reading the apps on top, and the contract stops a new shortcut the day it's written.

Nomenclature

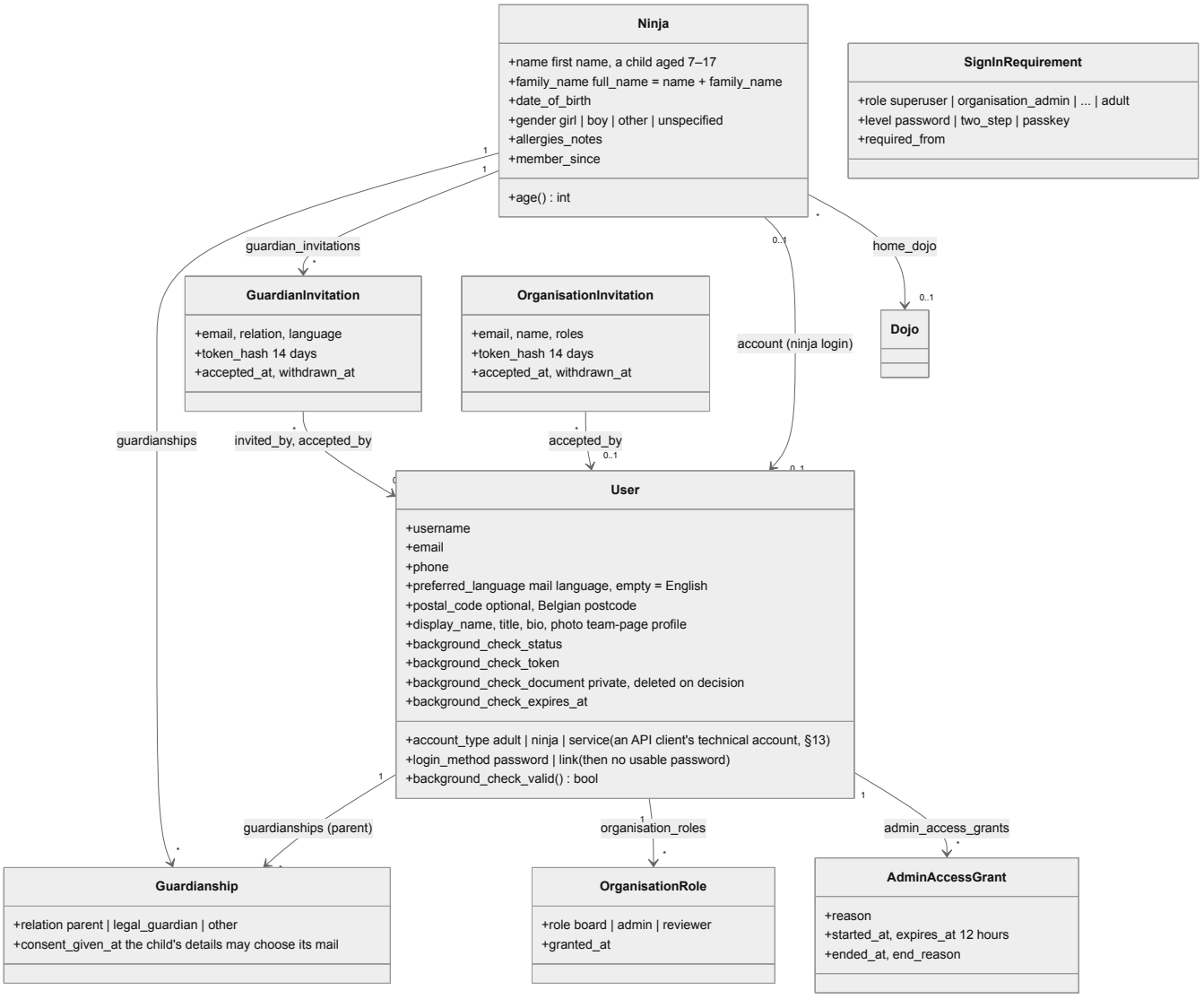
The code and the UI use CoderDojo's own vocabulary:

Term	Meaning	In the model
Champion	The dojo owner; exactly one per dojo	membership role `champion`
Mentor / Coach	An adult helper	membership role `mentor`
Ninja	A child aged 7–17 visiting a dojo	`Ninja`, plus an optional ninja login (`User`)
Youth mentor	A ninja helping run sessions	membership role `youth_mentor` on a ninja login
Badge	An award: one-off, or a milestone reached by a count	`Badge` (`one_off` / `milestone`), `NinjaBadge`
Belt	A ninja's proficiency level	`Belt`, the append-only `NinjaBelt` history

5 Accounts, families and roles

One `User` table is every login, told apart by `account_type`: `adult`, `ninja` (a child's own login) or `service` (an API client's technical account). There are no role subclasses.

- **Parents** are plain adult accounts linked to children through `Guardianship` (a child can have several guardians: a parent or legal guardian invites another adult, such as the child's other parent, by email, and every guardian is told of each change). No check, no approval: a parent only ever manages their own children.
- **A `Ninja` is not a user.** It gets a login only if a guardian gives it one; most never do. The guardian creates and removes it and keeps editing the child's details.
- **Champions and mentors** are adult accounts with an approved `Application` and a valid background check; what they do at a dojo is a `DojoMembership`.
- **Organisation roles** (`board`, `admin`, `reviewer`) map to permission groups kept in sync by signals, and open areas of the organisation dashboard. The Django admin needs a separate, 12-hour grant.
- Every login chooses a **password or an emailed login link**, with optional two-step login on top.

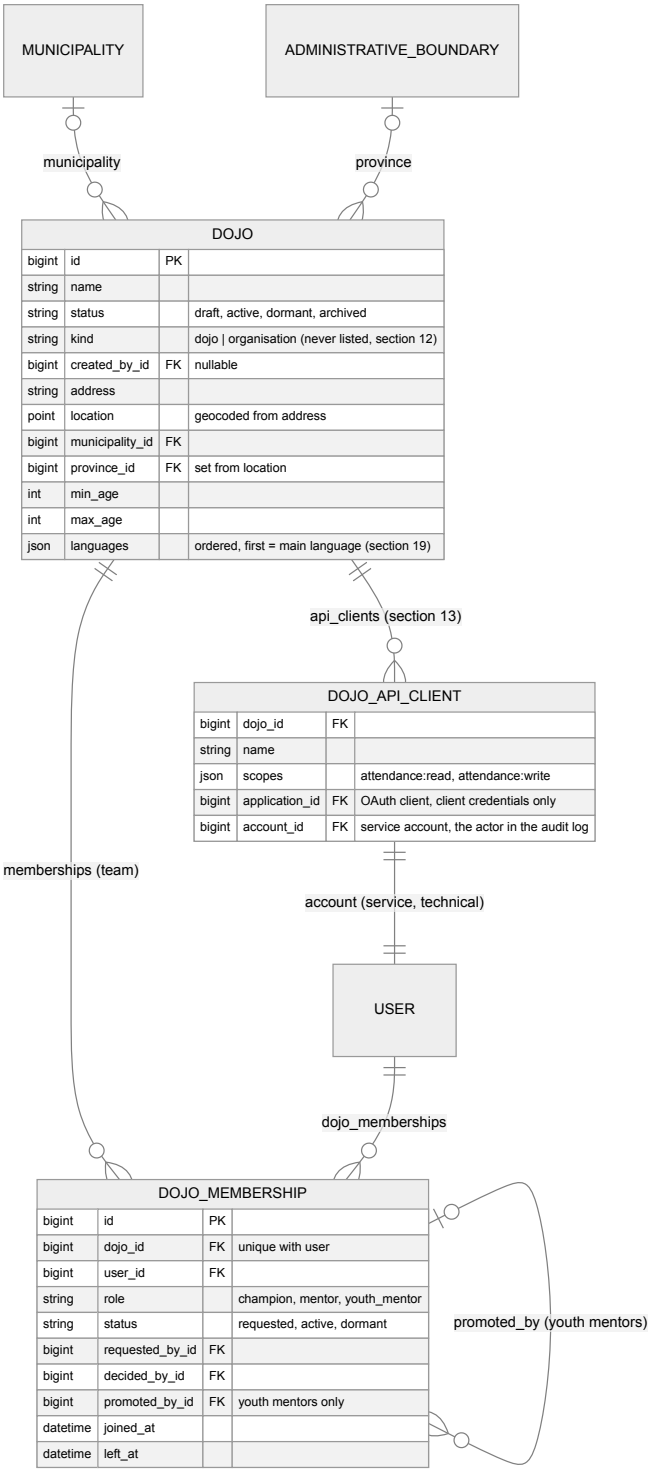


Accounts, guardianship and organisation access (DATA_MODEL.md §2).

Why. Being a parent, a mentor or a champion describes how a person relates to children or to a dojo, not what kind of account they have, and one person can be all three. Modelling roles as subclasses spread one idea over several tables and needed sync code; relations make each role a row that can be added, ended and audited.

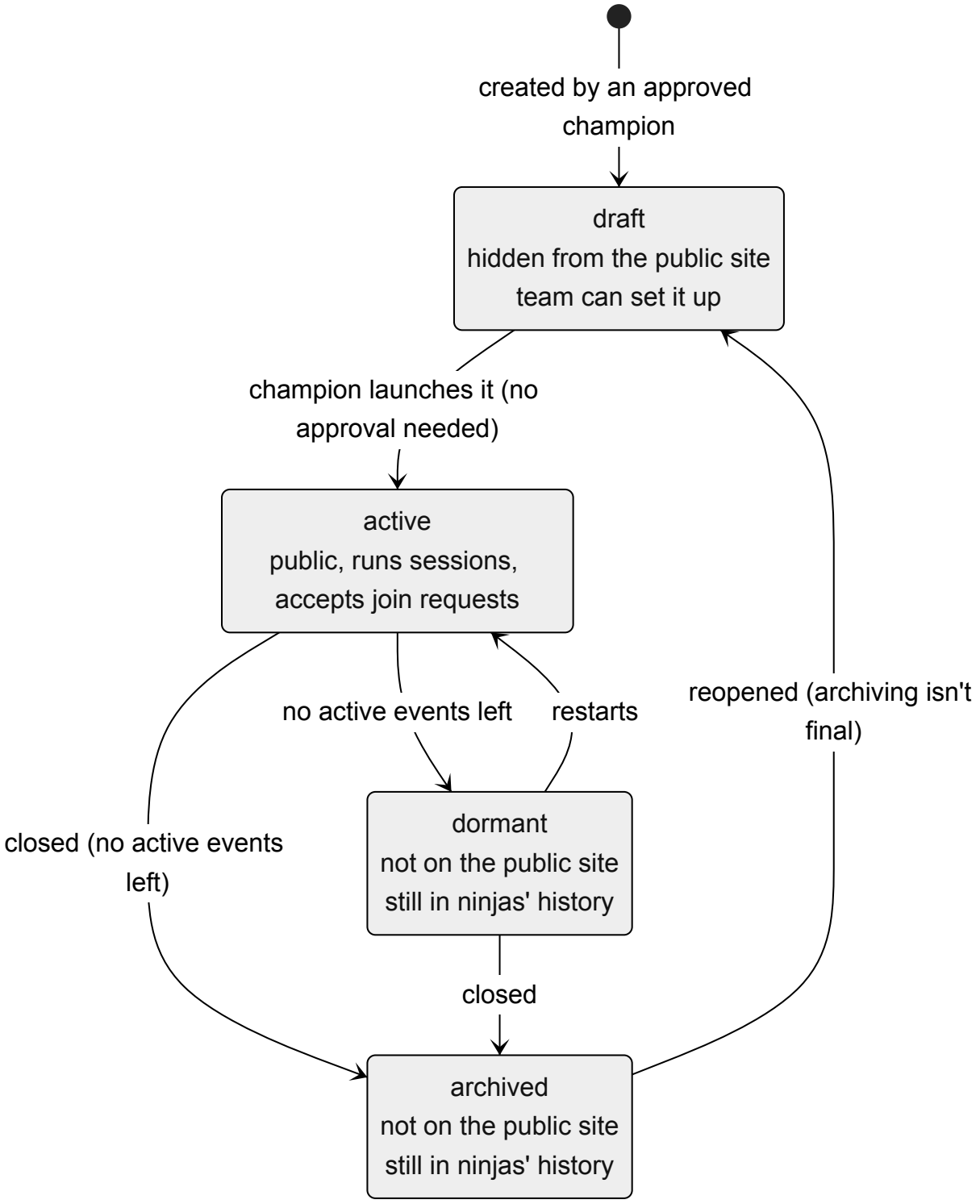
6 Dojos, teams and access

A dojo's team is a set of **memberships**, one row per account and dojo, each with a role and a status. The team-page profile (name, title, bio, photo) lives on the account and is shared by every dojo.

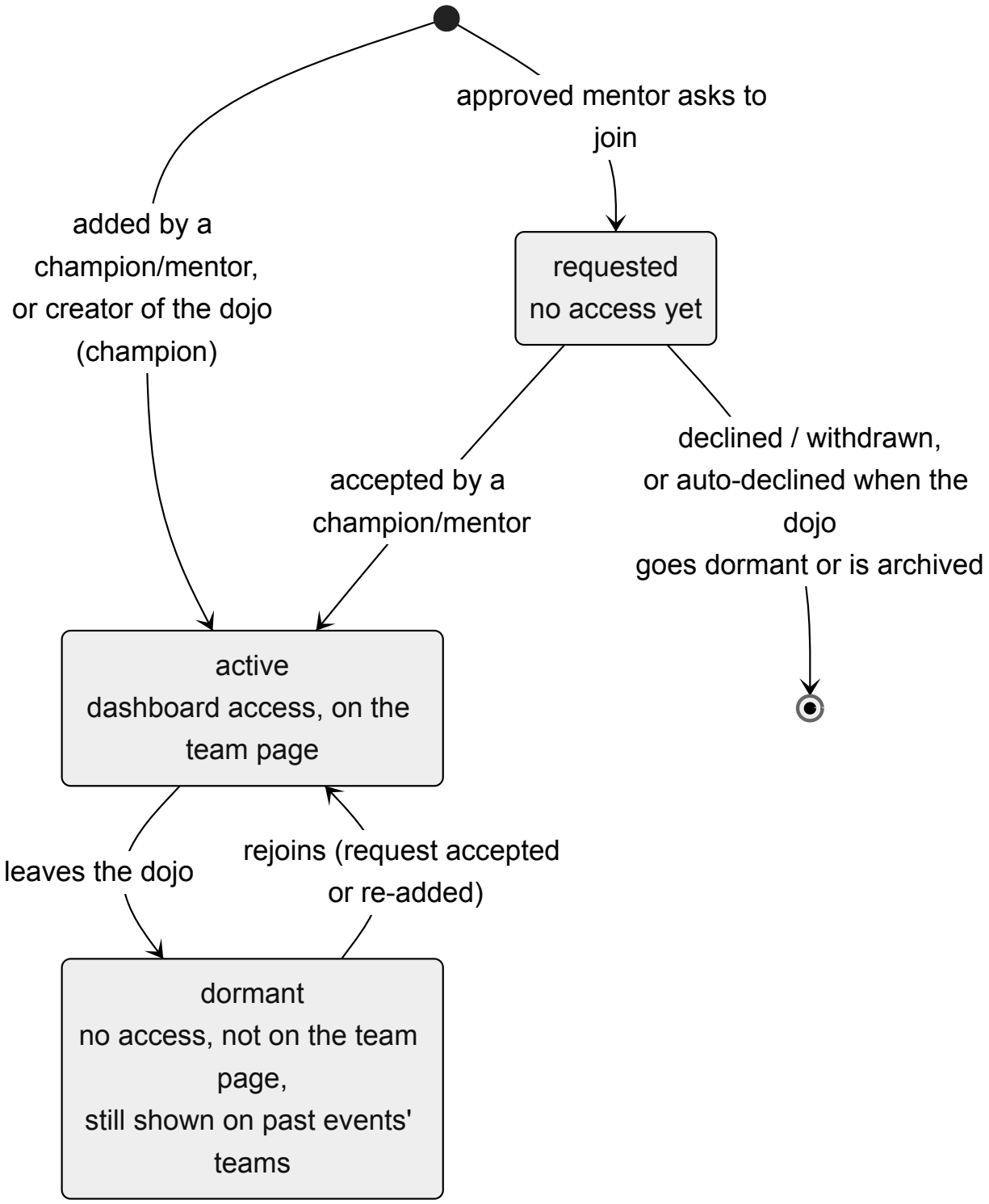


Dojos and their team memberships (DATA_MODEL.md §3).

Lifecycles

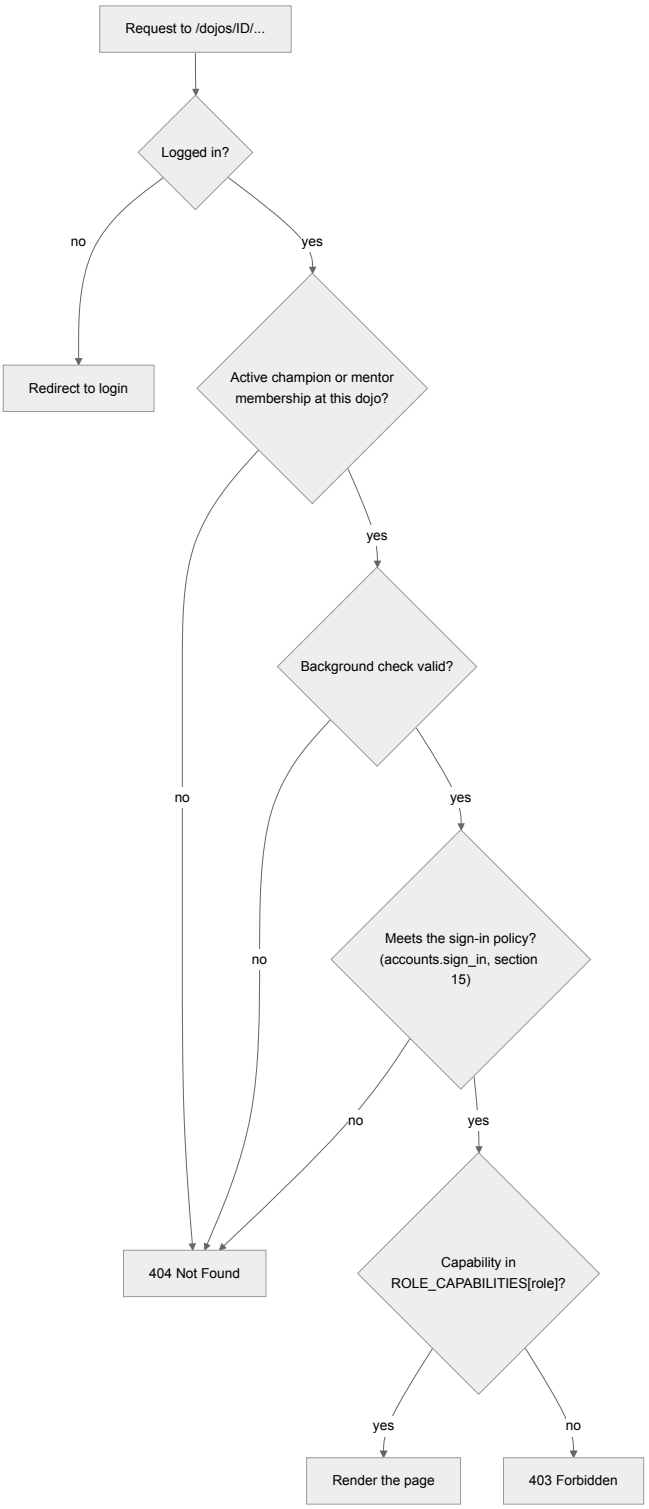


A dojo's status. Only active dojos (and their sessions) are public. Going dormant or archived needs no open sessions and declines pending join requests.



A team membership. Leaving makes it dormant, never deleted, so past sessions keep their team.

Who gets into a dojo's admin area



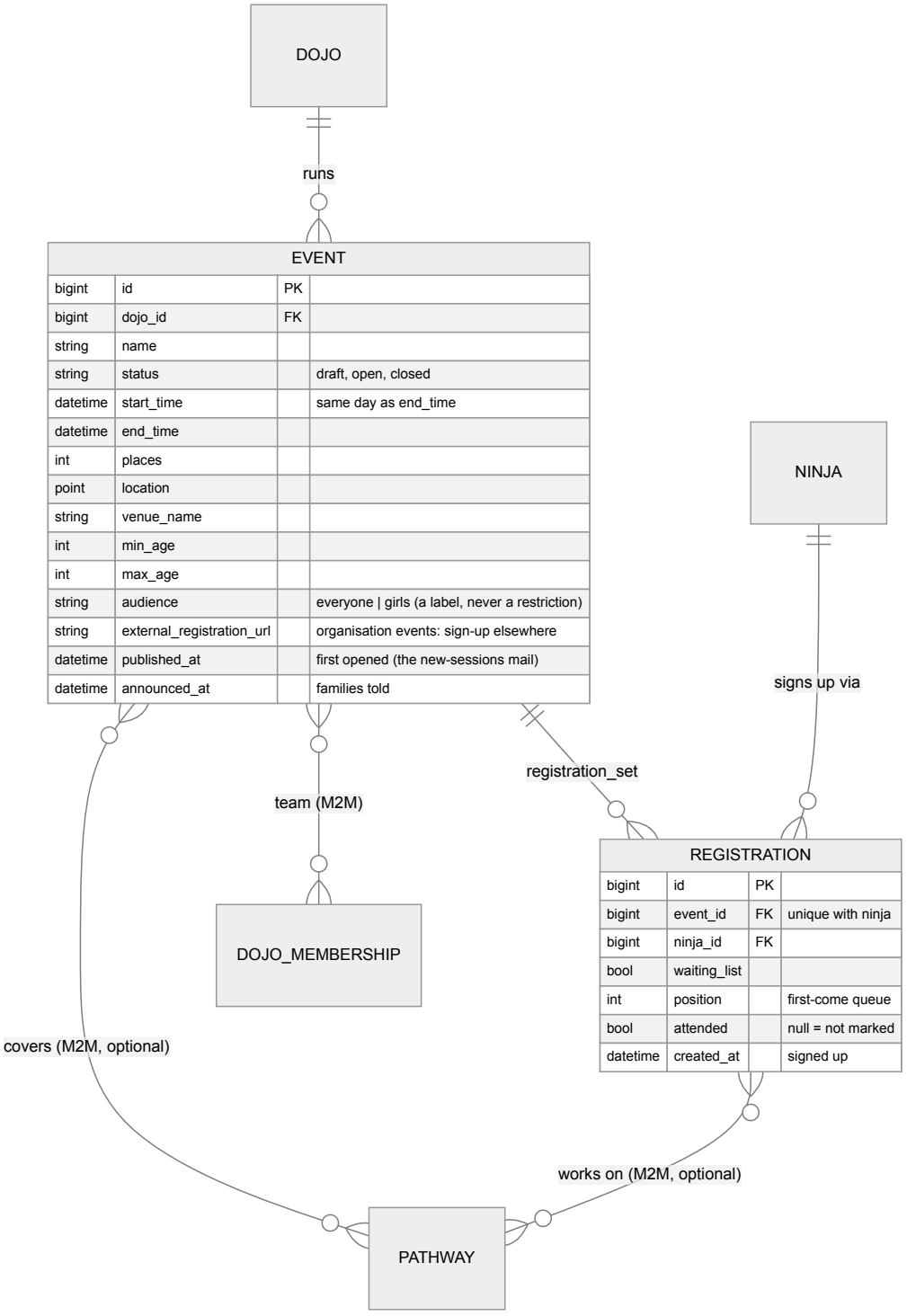
Access is always resolved through dojos.access.require_dojo_access.

Capability	Champion	Mentor
Dashboard, sessions list, the bell (any role)	✓	✓
`TAKE_ATTENDANCE`, `MANAGE_EVENTS`, `EDIT_SETTINGS`, `MANAGE_TEAM`	✓	✓
`AWARD_BELTS`, `AWARD_BADGES`, `POST_UPDATES`	✓	✓
`MANAGE_LIFECYCLE`, `VIEW_HEALTH_NOTES`, `MANAGE_API`, `SEND_MAIL`	✓	—
Hand over the champion role	✓	—

> **Why.** A lapsed background check never blocks login: the person can still use the site as a parent, but their memberships stop granting access until a renewal is validated. The membership keeps its status, so nothing needs repairing afterwards.

Organisation-run events (CoderDojo Girlz, Coolest Projects) live on an **organisation dojo** (`Dojo.kind = organisation`): a normal dojo with a team and an admin area, never listed publicly, and the only kind whose events may link out to an external registration site.

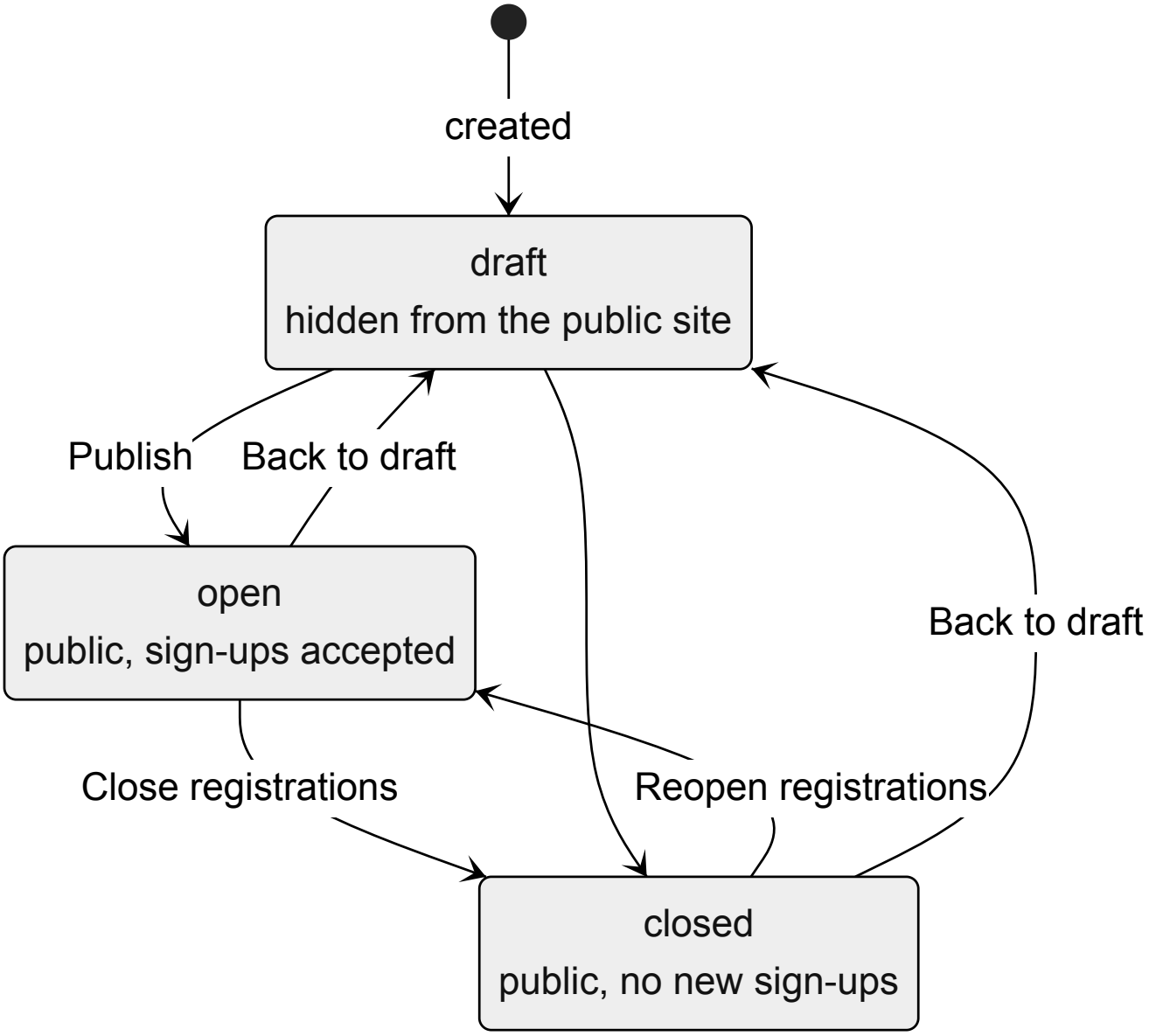
7 Sessions, registrations and attendance



Events, registrations, pathways and the session's team (DATA_MODEL.md §4).

- **Places and the waiting list.** Places left = places − confirmed registrations. Cancelling deletes the registration (after logging a `RegistrationCancellation`) and promotes the first child waiting; that family is mailed and the dojo's team notified.
- **Attendance is tri-state** (`None` = not marked, present, absent), for children and for the session's team (`TeamAttendance`, the record the insurance needs).
- **A girls' session** (`Event.audience`) is a label and a targeting signal, never a sign-up restriction.

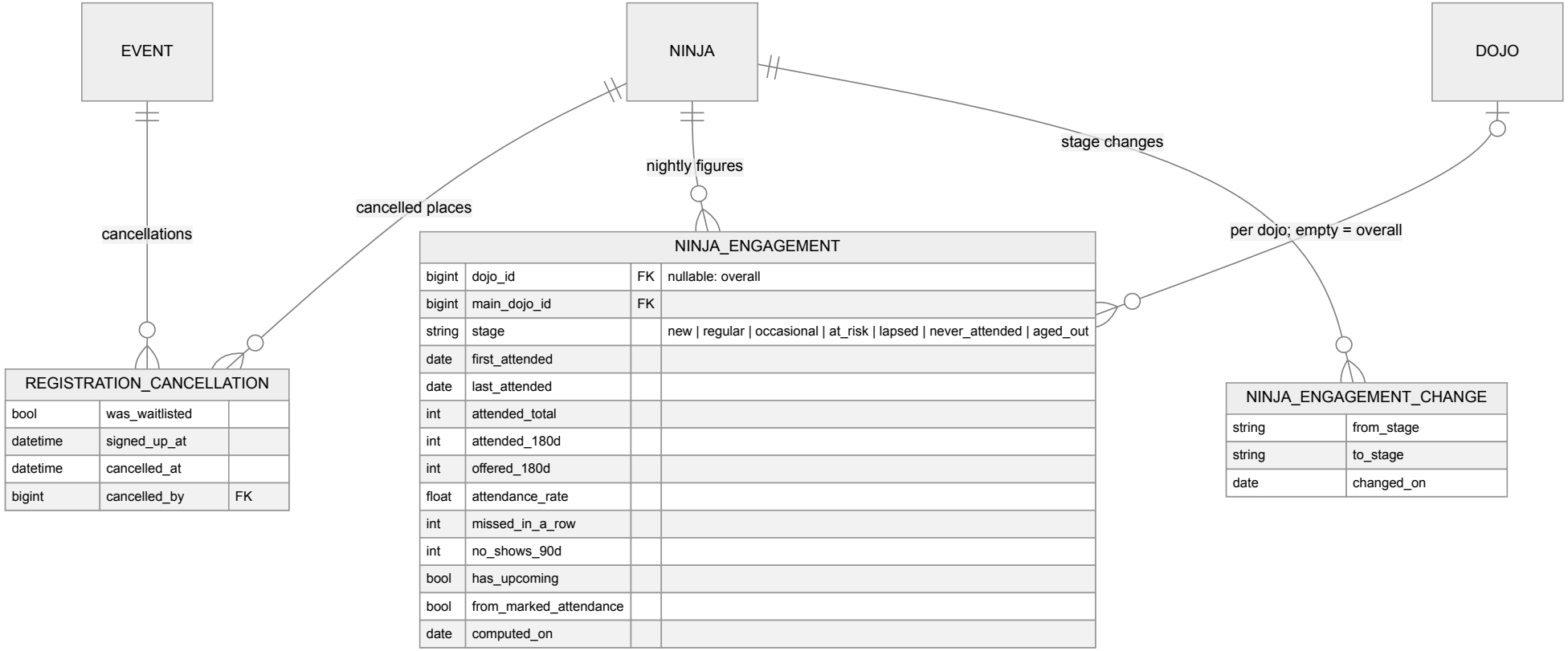
- **Home dojo:** set at a child's first sign-up, then only the guardian changes it.



A session's status is set directly by the team; it isn't one-way, and registrations are always kept.

Engagement

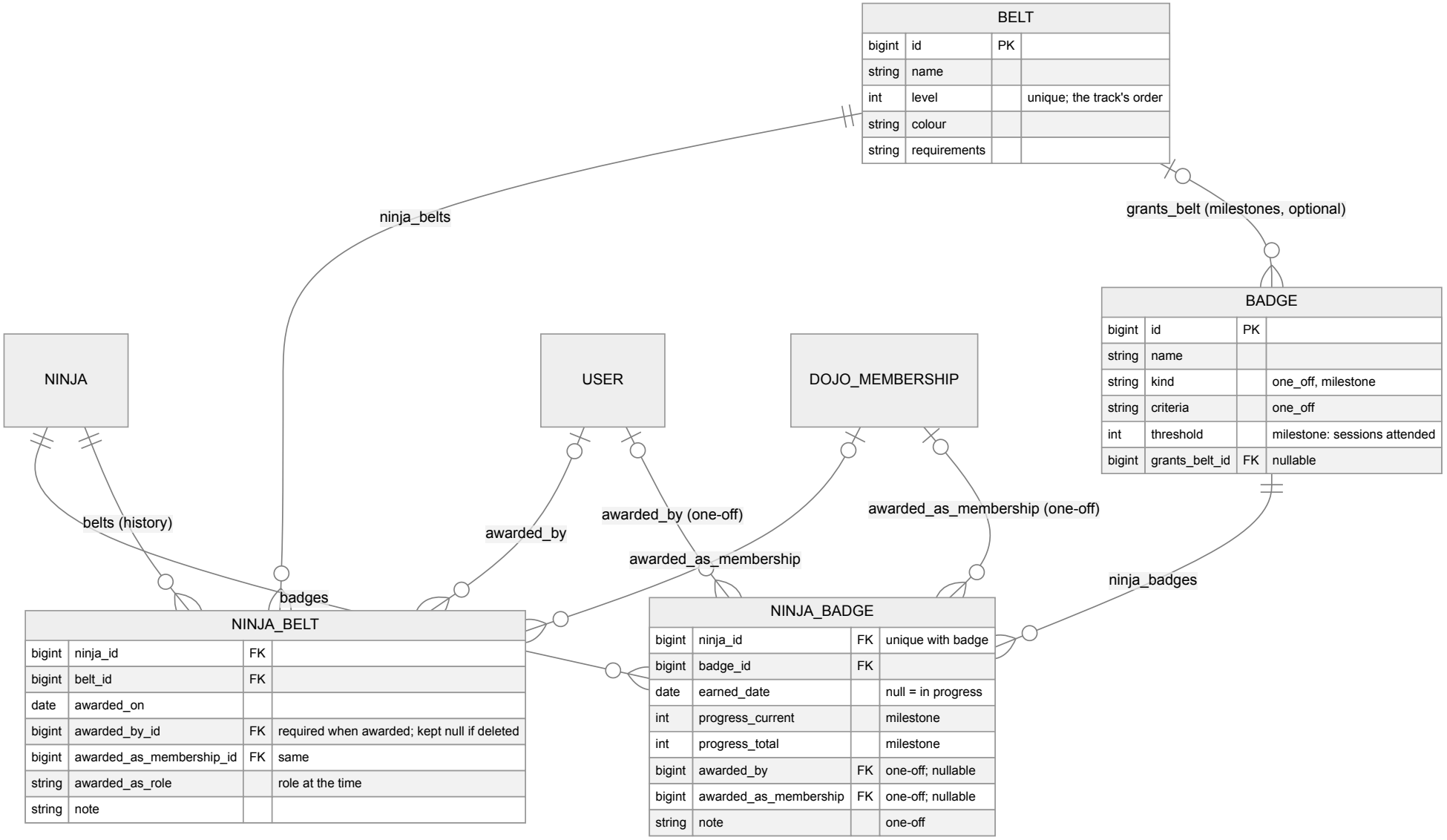
Every night `events.engagement.rebuild()` recomputes, per child and dojo, how they come to sessions: new, regular, occasional, at risk, lapsed, never came or aged out. Only sessions meant for the child count (age range, girls' sessions for girls). Stage changes are logged, which lets a *journey* send a "we miss you" mail the week a child becomes at risk.



Cancellations and the nightly engagement snapshot.

8 Badges and belts

- A **badge** is an award. *One-off* badges are awarded by a dojo's team from the attendance list; *milestone* badges (the attendance wristbands) are recomputed whenever attendance is marked and stay earned. Only the organisation defines badges.
- A **belt** is a ninja's proficiency level on one track, the only record of their coding level. The history is append-only; the current belt is the highest. Only an active champion or mentor awards one, recording the account *and* the membership (“Jan, as mentor of Dojo Ghent”). A milestone badge can grant a belt.

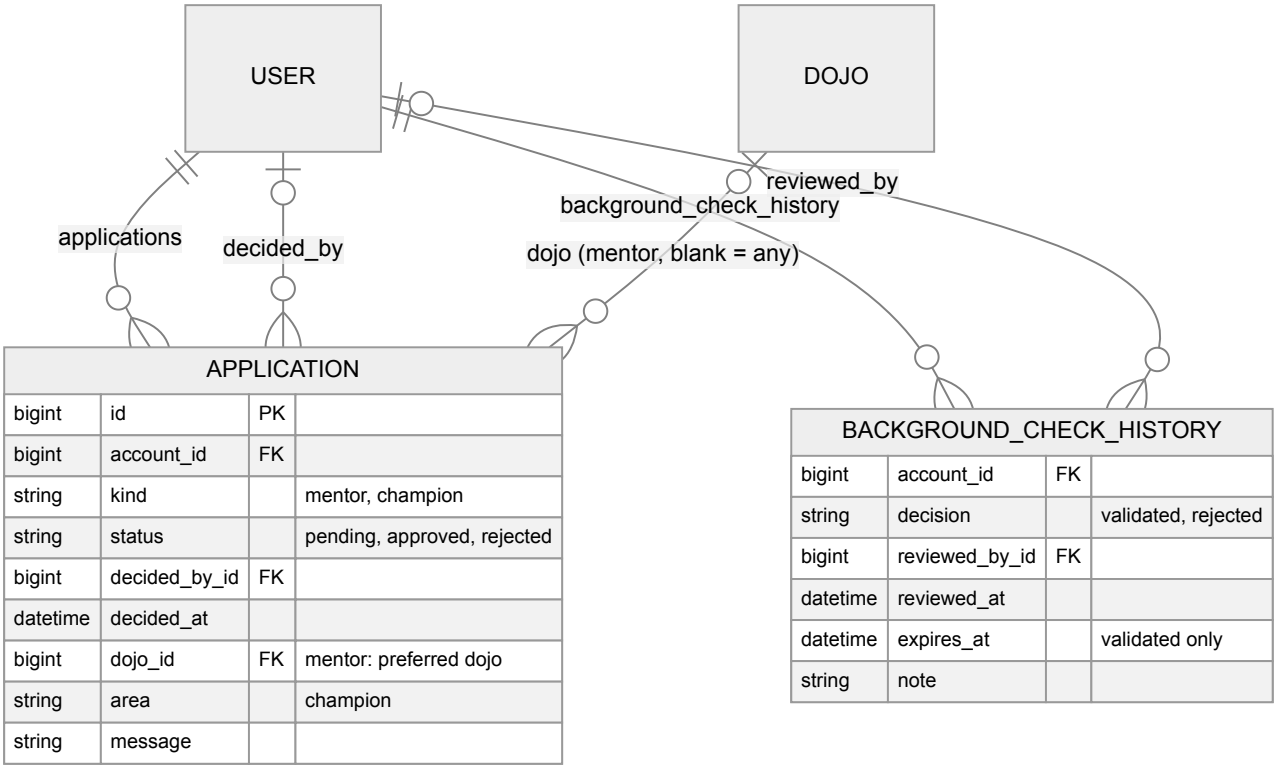


Badges and belts; all rules in events/awards.py (DATA_MODEL.md §5).

Why. Attendance and skill are different things: a child who comes every week isn't necessarily an advanced coder. Keeping milestones as badges and belts as a separate level lets both be shown honestly.

9 Volunteers: applications and background checks

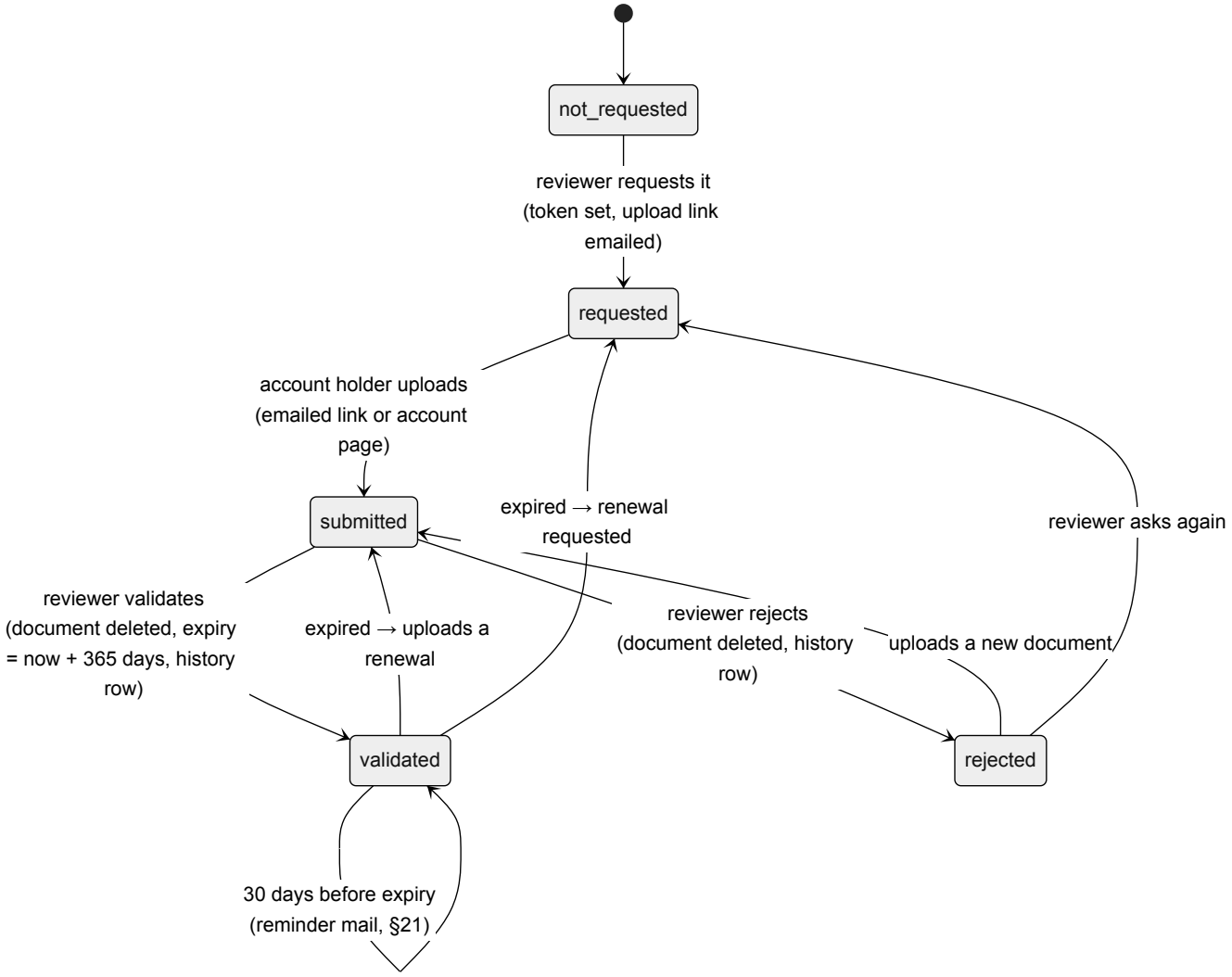
Belgian law (Art. 596.2) requires a criminal-record extract, model 2, for anyone working with minors. Onboarding is **per account, once**: an approved mentor can join any number of dojos, an approved champion can create one.



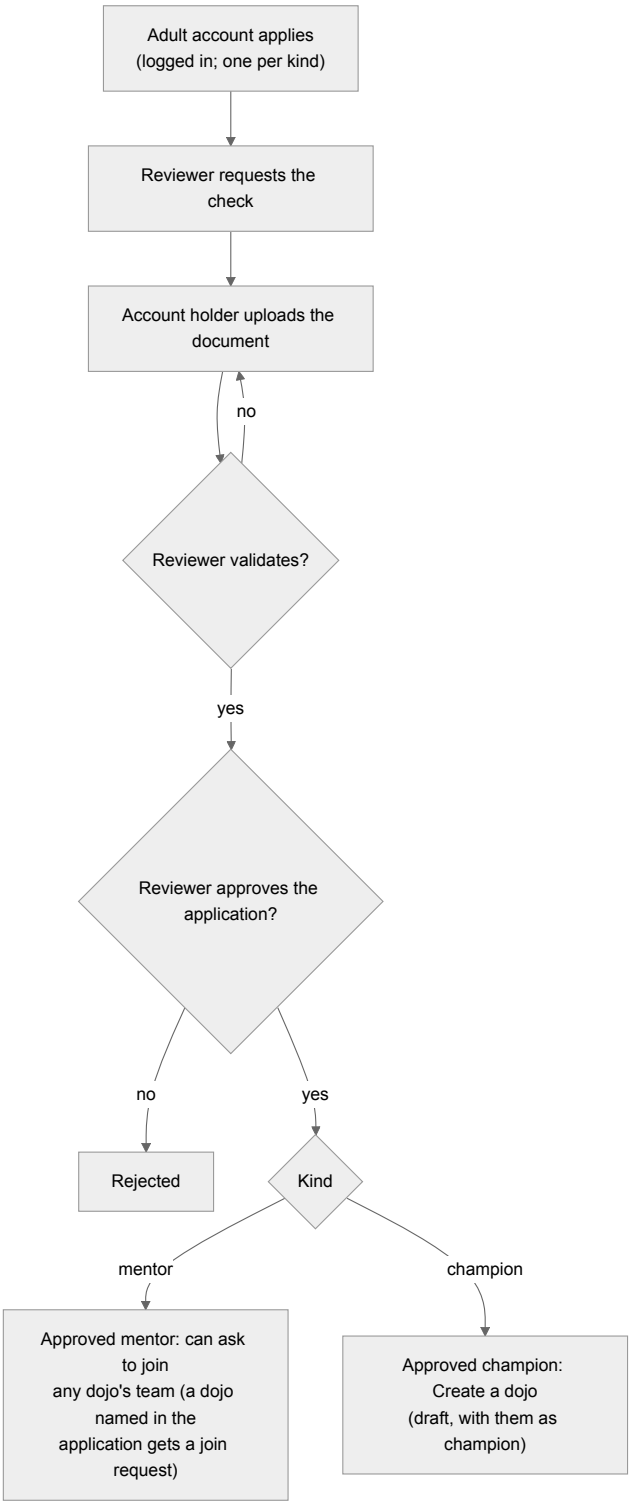
APPLICATION			
bigint	id	PK	
bigint	account_id	FK	
string	kind		mentor, champion
string	status		pending, approved, rejected
bigint	decided_by_id	FK	
datetime	decided_at		
bigint	dojo_id	FK	mentor: preferred dojo
string	area		champion
string	message		

BACKGROUND_CHECK_HISTORY			
bigint	account_id	FK	
string	decision		validated, rejected
bigint	reviewed_by_id	FK	
datetime	reviewed_at		
datetime	expires_at		validated only
string	note		

Applications and the append-only decision history (DATA_MODEL.md §6).



The background check lives on the account. Either decision deletes the document.



From application to a dojo.

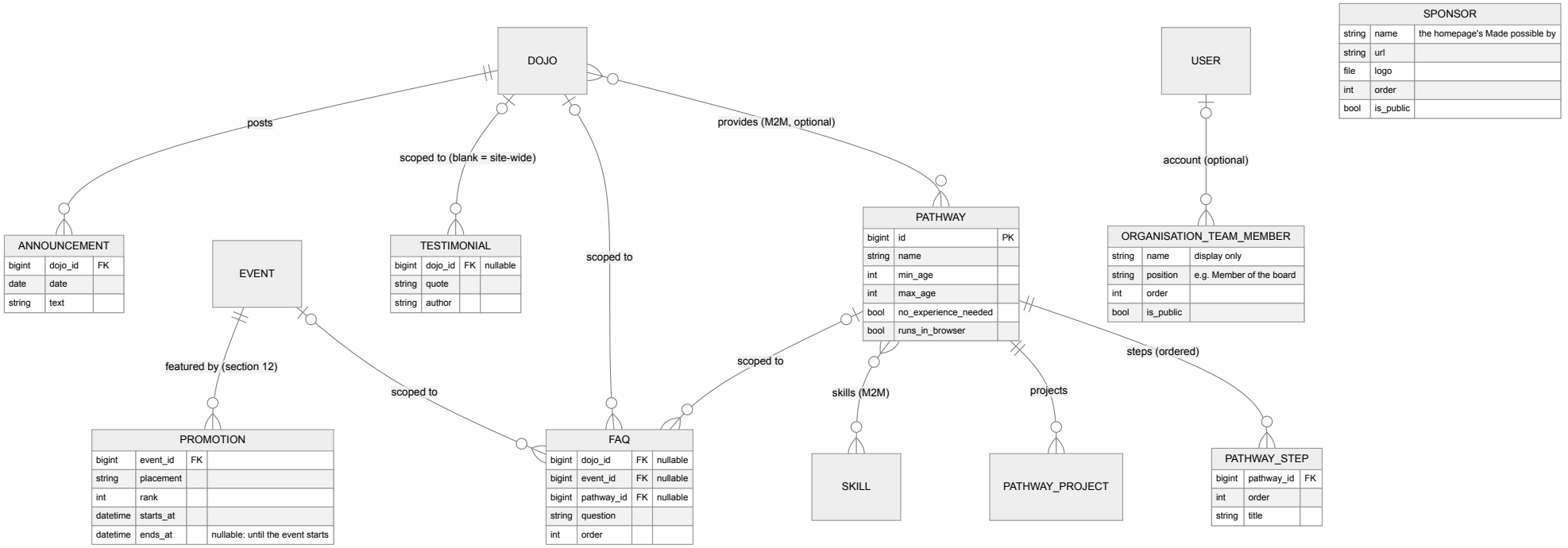
> **Why.** A check is about a person, not about a dojo. Per-dojo applications made volunteers repeat the process and scattered the check across rows. Keeping only the decision (who, when, until when) and deleting the extract itself keeps the most sensitive document the site ever sees for the shortest possible time.

Reviewers work on the organisation dashboard's *Volunteers* pages; nobody decides on their own check or application. The account holder is reminded 30 days before expiry, and reviewers get a daily mail while documents wait.

10 Pathways, content, notifications and geography

Pathways and content

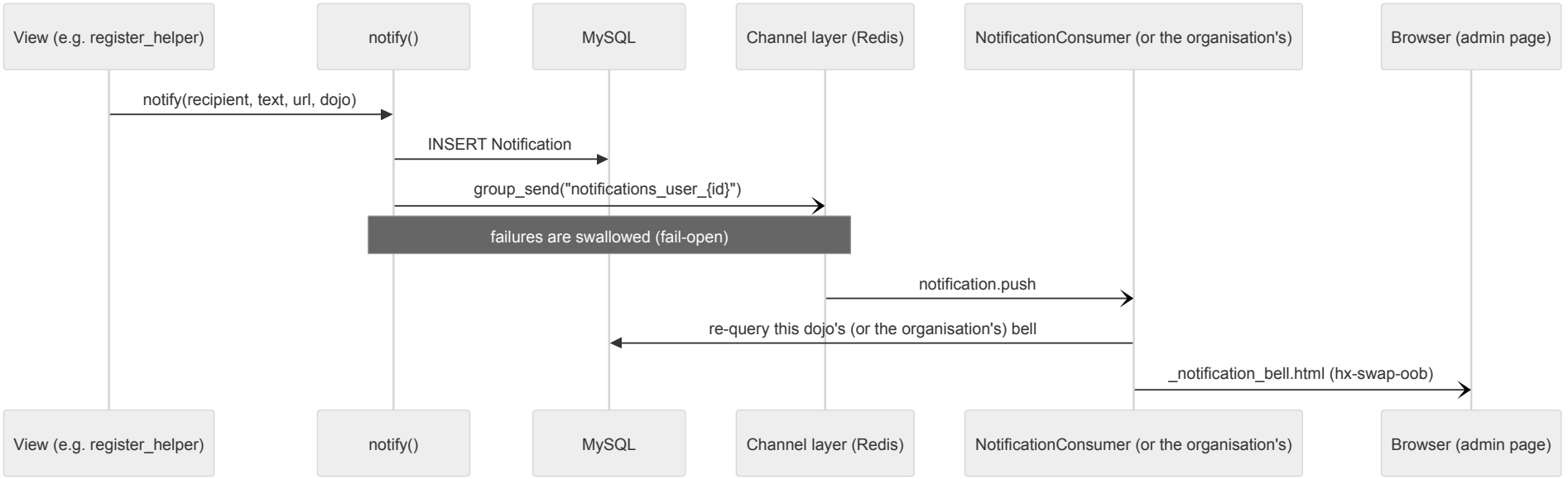
Pathways are a read-mostly catalogue, linked at three optional levels, each pre-filled from the one above but never restricted to it: what a dojo provides → what a session covers → what one ninja works on.



Pathways and the content models; content is scoped to a dojo, event or pathway, or site-wide (DATA_MODEL.md §7).

Notifications

One row per recipient, so read state is per person. `notify()` writes the row (the source of truth), then nudges the recipient's open pages over the Channels layer; a failure there is swallowed. Texts are stored as translatable messages and rendered in the *recipient's* language.



A live notification: the consumer re-renders the bell and htmx swaps it in.

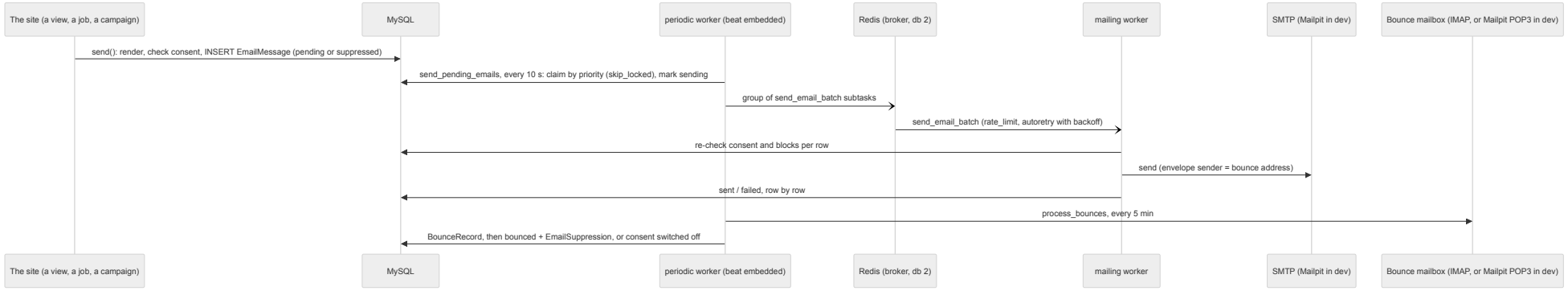
Geography

Municipalities (postcodes with a centre) and administrative boundaries are reference data. A dojo's province is derived from its geocoded location (point in polygon, nearest boundary as fallback). The dojo finder orders by `DistanceSphere` from a typed address, the browser's location, the family's postcode or a Ghent default; only the default list is cached, so one family's results never leak to another visitor.

A typed address becomes coordinates through `geo.geocoding.geocode`, the only way to call OpenStreetMap's Nominatim, which allows one request a second for the whole site. It answers from the cache first (90 days for a match; the key is a hash, so what someone typed isn't stored), then from our own municipalities for a postcode or a town's name, and only then asks Nominatim, after taking a site-wide slot in Redis. When Nominatim asks us to slow down, every call stops for a while; without Redis there's no call at all.

11 The mail engine

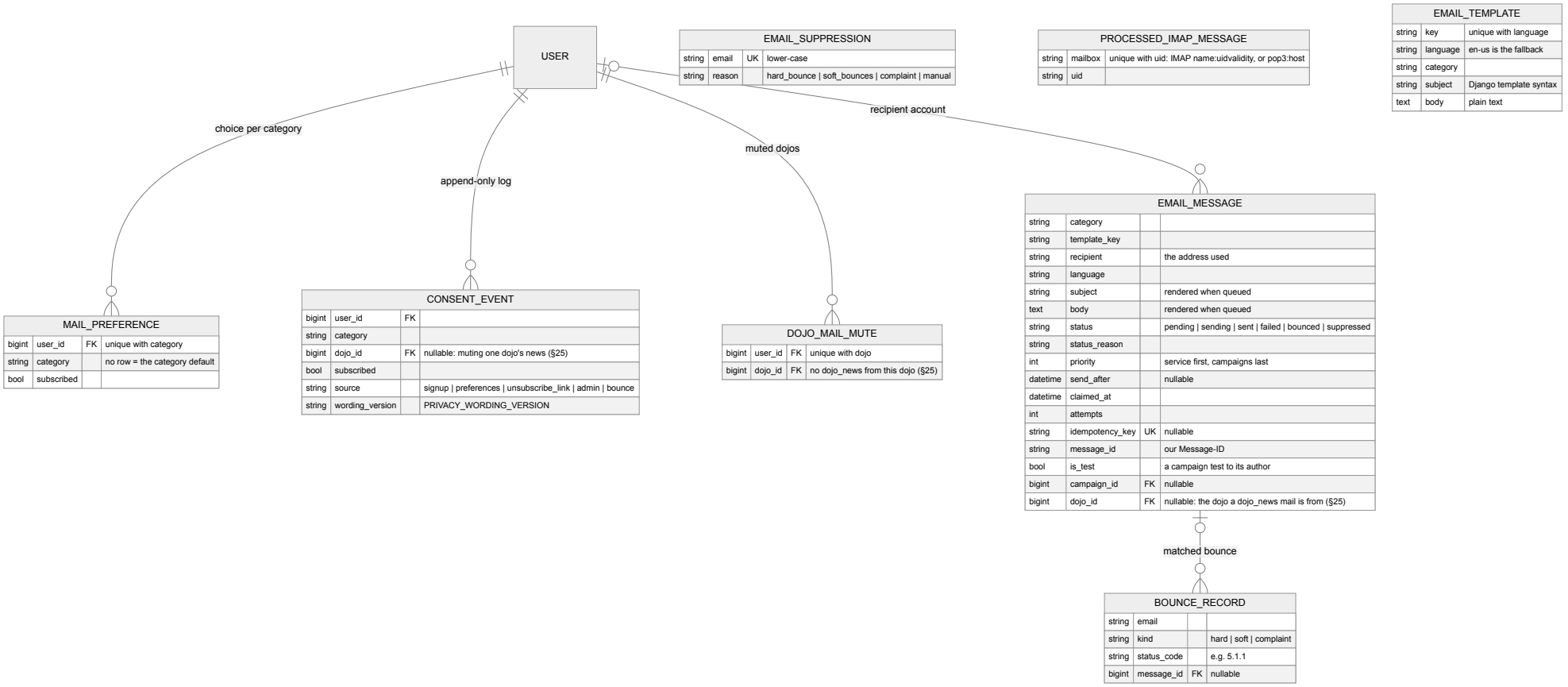
Every mail goes through one gateway, `mailing.services.send()`: it renders the template in the recipient's language, checks consent and blocks, and inserts an `EmailMessage` row. **The database is the queue**; the Celery workers only send.



How a mail goes out (DATA_MODEL.md §11).

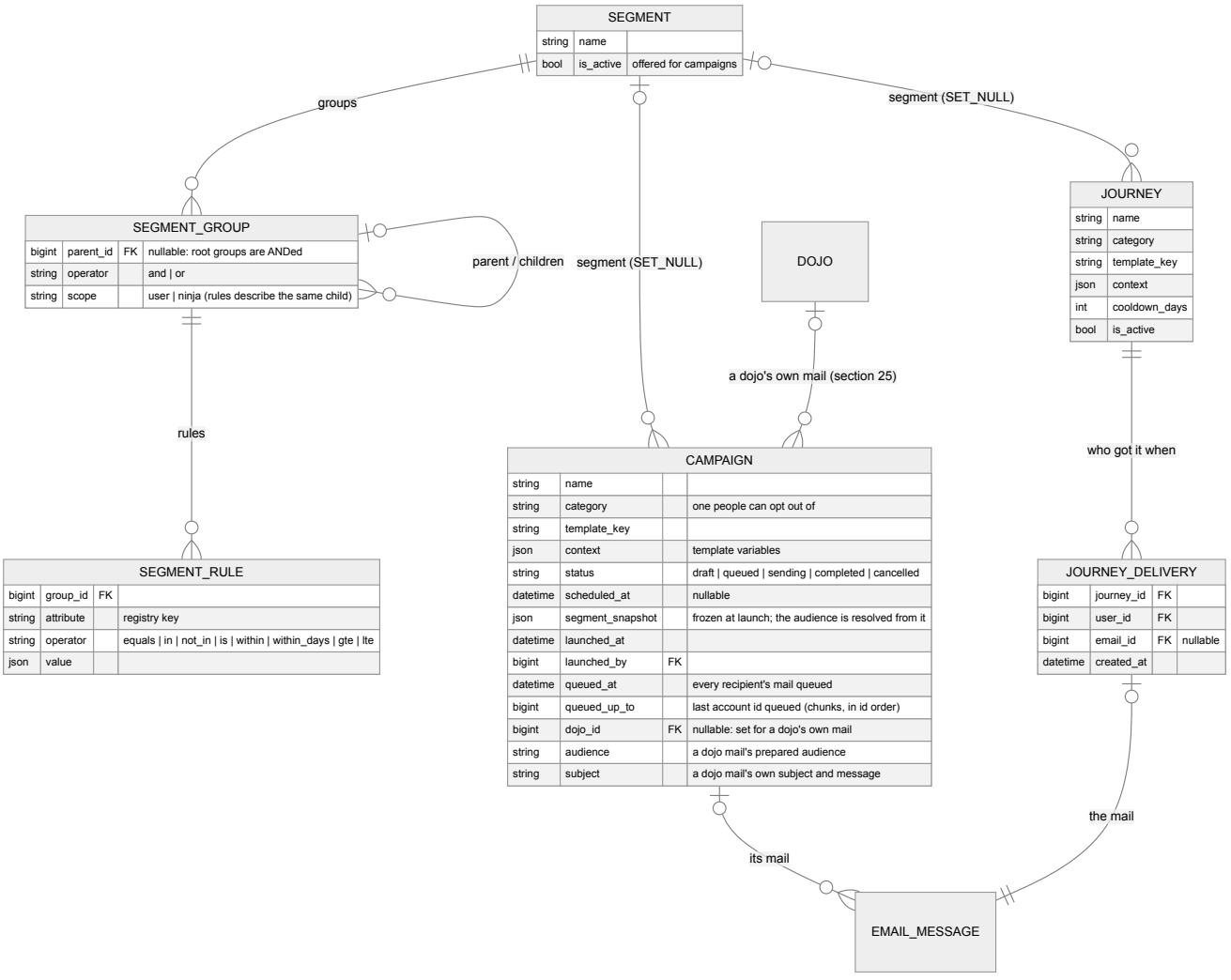
> **Why.** A row per mail is the record of exactly what was sent, survives a broker restart, and makes retries and idempotency simple: a row is marked sent right after its own send, so a retry never mails anyone twice.

Worker	Runs	Why separate
`periodic` (beat embedded)	The 10-second dispatcher, requeueing, bounces, the jobs beat triggers	The dispatcher never waits behind a big campaign; exactly one beat.
`mailing` (default queue)	Batches of mail, campaign launches, everything else	With one process, Celery's rate limit is the real limit towards SMTP.



Consent and preferences, the queue, blocks and bounces.

- **Categories** (service, registration, reminder, dojo news, volunteer, newsletter) each with a default and whether people may opt out; `ConsentEvent` logs every change with the privacy wording's version.
- **Segments** describe an audience without code: nested groups of rules about accounts or about “the same child”; each rule is its own subquery. Child rules only reach guardians who consented.
- **Campaigns** freeze their segment at launch; **journeys** are standing campaigns run daily with a cool-down.
- **Bounces** are read from a mailbox (DSN, ARF): a hard bounce blocks the address, a complaint switches off optional mail.
- **A dojo's own mail** is a campaign with a dojo, sent by its champion to prepared audiences, with Reply-To the dojo.

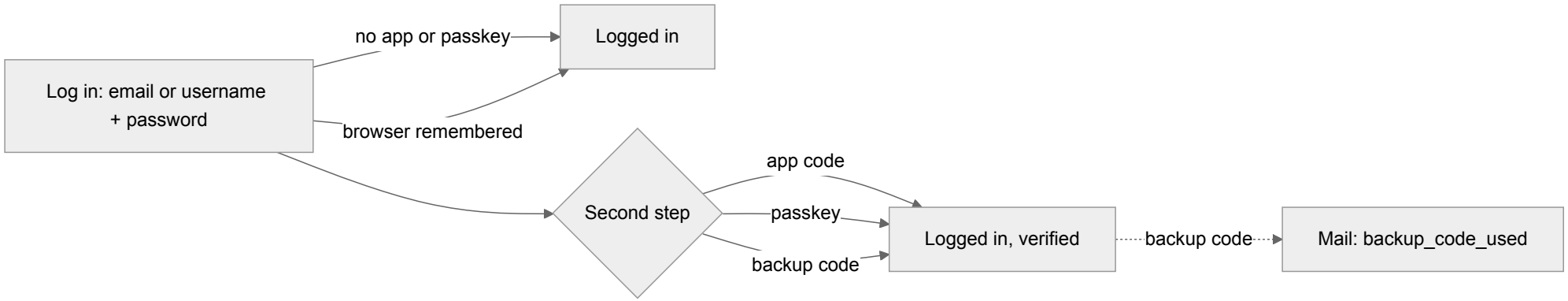


Campaigns, journeys and segments.

12 Security

Logging in

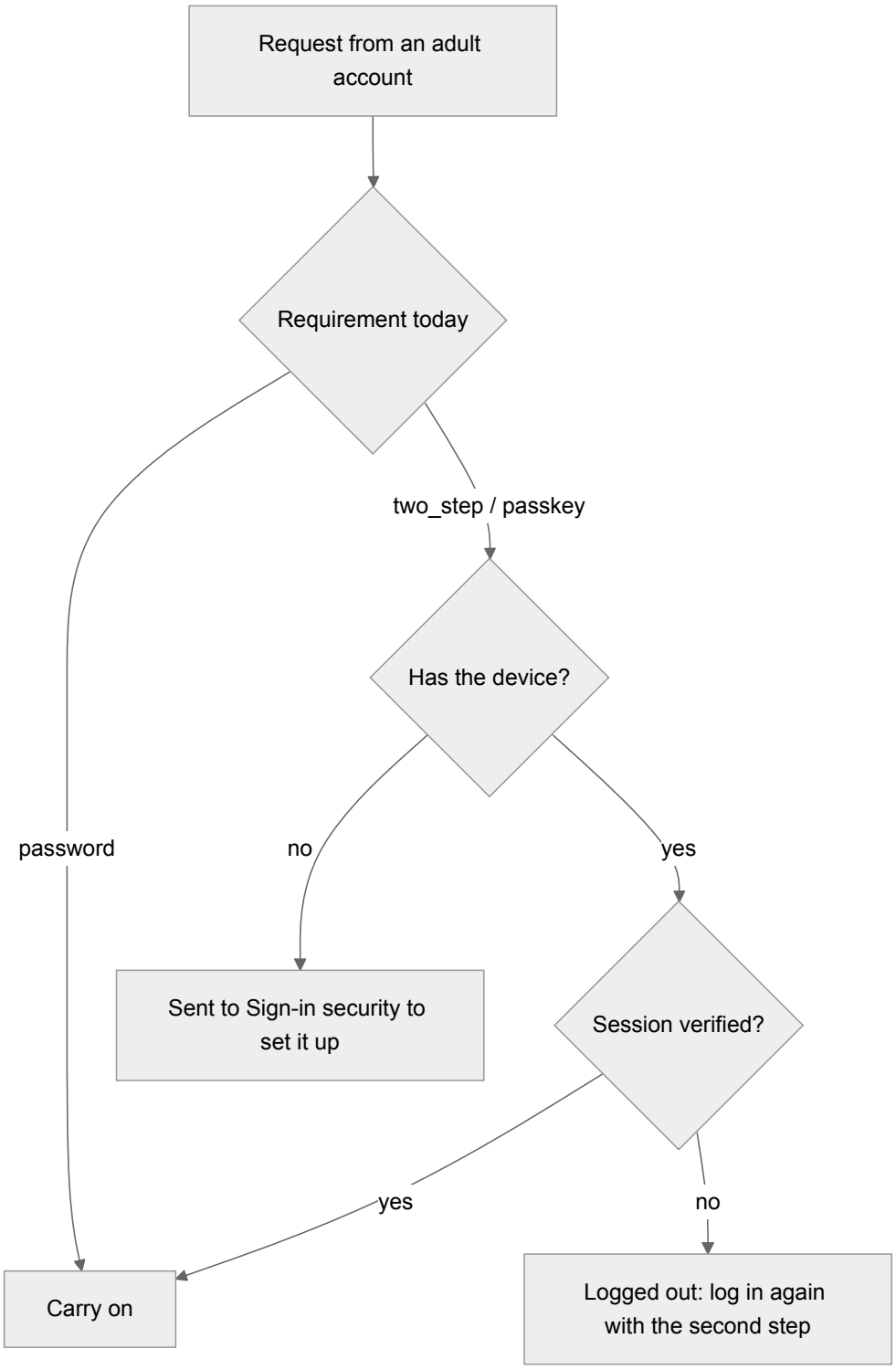
There is one login page. A login link is the same login view with a different first step, so the second step always follows. Passkeys are checked against the site's own URL, since TLS ends at the proxy.



Password (or login link), then the second step when the account has one (DATA_MODEL.md §15).

The sign-in policy

The organisation sets, per role and from a chosen date, the minimum: password, two-step login or passkey. An account's requirement is the strongest among its roles. Today every role is on “password”; the enforcement is in place for when that changes.



Enforcement: a middleware guides, and every door a role opens checks again.

Access to the management side

- The organisation dashboard is split into **areas** (communication, public site, ninjas, privacy, security, volunteers, audit log), each opened by a permission the roles grant.
- The **Django admin** needs a grant asked for with a reason and the password, for 12 hours; the other admins are notified. Superusers are only made on the server.
- The **API** gives an app one dojo's data with scoped OAuth 2.0 tokens, through its own technical account, and is tested never to expose a field classified as special, criminal or security data.

13 Privacy by design

Most of the data is about **children**, one field is **health data** (allergies and notes, GDPR art. 9) and one flow handles **criminal-record extracts** (art. 10). Privacy is therefore built into the model rather than bolted on.

Every field is classified

Each app declares its models in a `privacy.py`: purpose, legal basis, retention, who sees it, and for every field whether it is personal (and in which category) and what happens to it on erasure: emptied, anonymised with a replacement, or kept with a reason. A test fails on any unclassified field, ours or third-party. From that one registry:

- **The register of processing activities** (`manage.py privacy_register`).
- **A person's export** (art. 15/20), for families from their account page and for the organisation on request.
- **Erasure**: rows are found by the registry's subjects, fields emptied or anonymised, `User` and `Ninja` rows anonymised, never deleted, and an `ErasureRecord` without personal data allows replaying erasures after a backup restore.
- **Retention**: an account is erased two years after its last login (a parent's also counts their children's own logins and dojo sessions), after reminder mails; champions and mentors keep their public team profile. A mail's subject, body and address are cleared 12 months after it was queued; the row stays for the statistics.

The audit log

django-auditlog records changes to the models listed in `core.audit.RECORDED` (every other model has a written reason why not) and views of special-category data. Health notes and secrets are masked; IP addresses are never stored. It is read-only, for the organisation's admins, and its entries follow the same retention and erasure.

Who sees the most sensitive data

Data	Seen by	Safeguard
A child's health notes	The dojo's champion, only for children with a confirmed place	Every view recorded in the audit log
Criminal-record extract	Reviewers, until they decide	No URL; deleted on decision; downloads recorded
Families' addresses	Never the dojo team	Dojo mail shows counts only

14 Languages, testing and operations

Three languages, two kinds of text

- **The site's own texts** (templates, forms, messages, `bundle.js`) are in gettext catalogs for Dutch and French; Dutch uses the informal “je”, French “vous”. The site language follows a cookie; an account's `preferred_language` is only its mail language.
- **Content** written by people (a dojo's description, session names, updates, the organisation's pathways and badges) is stored per language on the row, in the languages the dojo (or the organisation) chose; visitors see the main language with an “Only in …” note when theirs is missing.
- Mail templates are rows per language, falling back to English.

Testing

- About 1,190 tests with the plain Django runner, inside the devcontainer, on their own Redis cache database.
- Route tests check status codes, templates, permission gating (404 versus 403) and the database effect of a POST.
- Channels consumers are tested with `TransactionTestCase` and an in-memory layer; Celery tasks by calling the function, never `.delay()`.
- The guard tests of chapter 3 keep new code honest about privacy, auditing, the admin, translations and the API schema.

Operations

- One deploy script, with a preflight and a `manage.py check` before the live app is touched.
- `/health/` for an external uptime monitor (database, Redis, the mail workers), and `/metrics/` (behind a token) for Prometheus: request and task timings, queues, table sizes and memory, never anyone's data.
- Mail waits safely in the database when the workers are down; the dashboard's mail queue warns when due mail has waited over 30 minutes.
- Seed commands build a realistic demo world (dojos, families, volunteers, histories, waiting lists) and a credentials file describing what each login can do; the user-journey PDFs are made from it.

Known gaps and open points

- The API covers dojo clients and attendance; external registrations and event management are planned (§13).
- Some privacy retention rules wait for their periods to be decided; their removals are built and stay off until then (§16).
- Production still lacks MySQL client headers and GDAL/GEOS on the host; the deploy script refuses to go live until then.
- Production's proxy still needs the 12 MB request-body limit the devcontainer's nginx has (chapter 15).

15 Capacity, load and disk

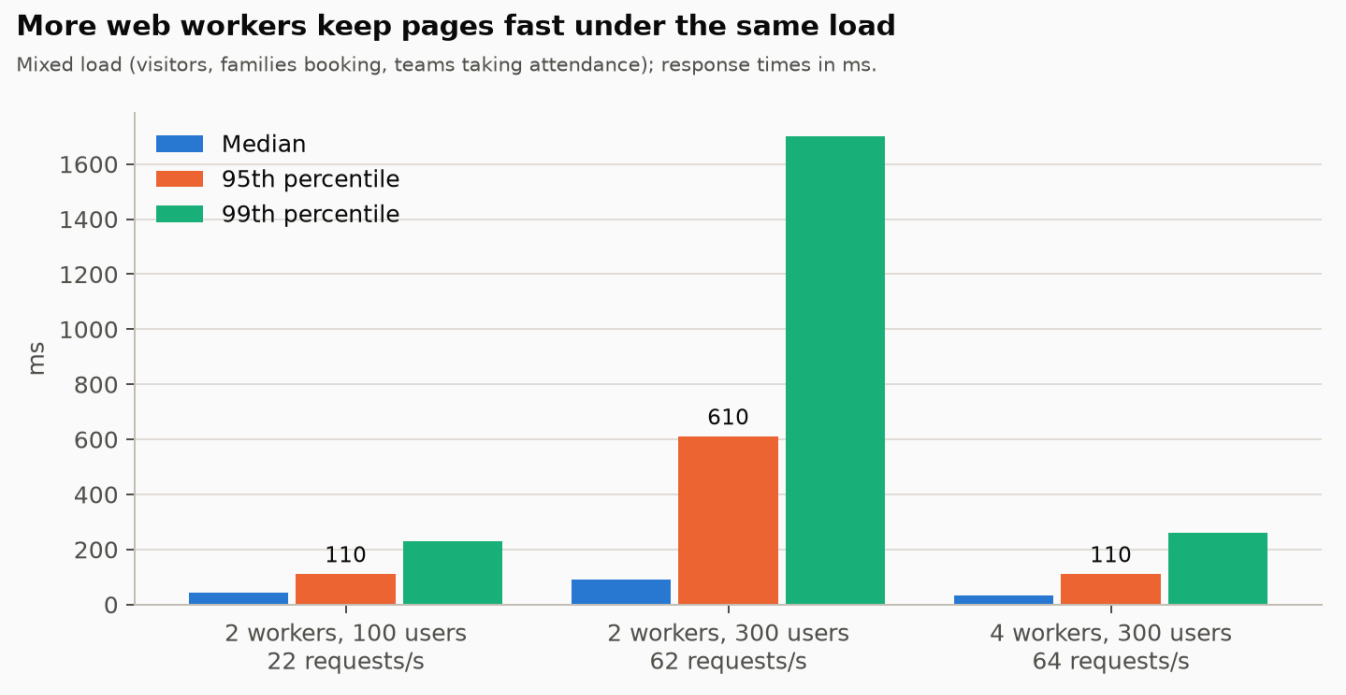
How big the database and the files get, what the site does under load, and how much memory each component needs: measured on 30 September 2026 in the devcontainer, on a database filled with a year of the *growth* scenario (100 dojos, 6,000 families, 1,200 sessions, 24,000 bookings and 189,000 mails a year). The full report, with the method and how to measure again, is `CAPACITY.md`; the charts come from `loadtest/charts.py`. The caching work was measured on 1 October 2026, the old and the new code with the same runs.

How it was measured

- `manage.py seed_scale` fills a separate `test_` database with a scenario's data, from rendered mail to the engagement snapshot; `capacity_report --measure` stores the bytes each row takes.
- A production-like run on it: `DEBUG` off, gunicorn with uvicorn workers and `gunicorn.conf.py`, the two Celery workers with production's options, on Redis databases of their own.
- Load from Locust (`loadtest/`): visitors, families that log in and book, dojo teams taking attendance, and a registration rush; the site's own `/metrics/` sampled every 2 seconds.
- Memory as PSS (each process's fair share of the memory forked processes share), never summed RSS.

> **Why.** The devcontainer isn't Level27's machine, so absolute response times will differ there. The memory figures and the comparisons between runs carry over; the public part of the test can be repeated on production itself.

Web workers set the response times



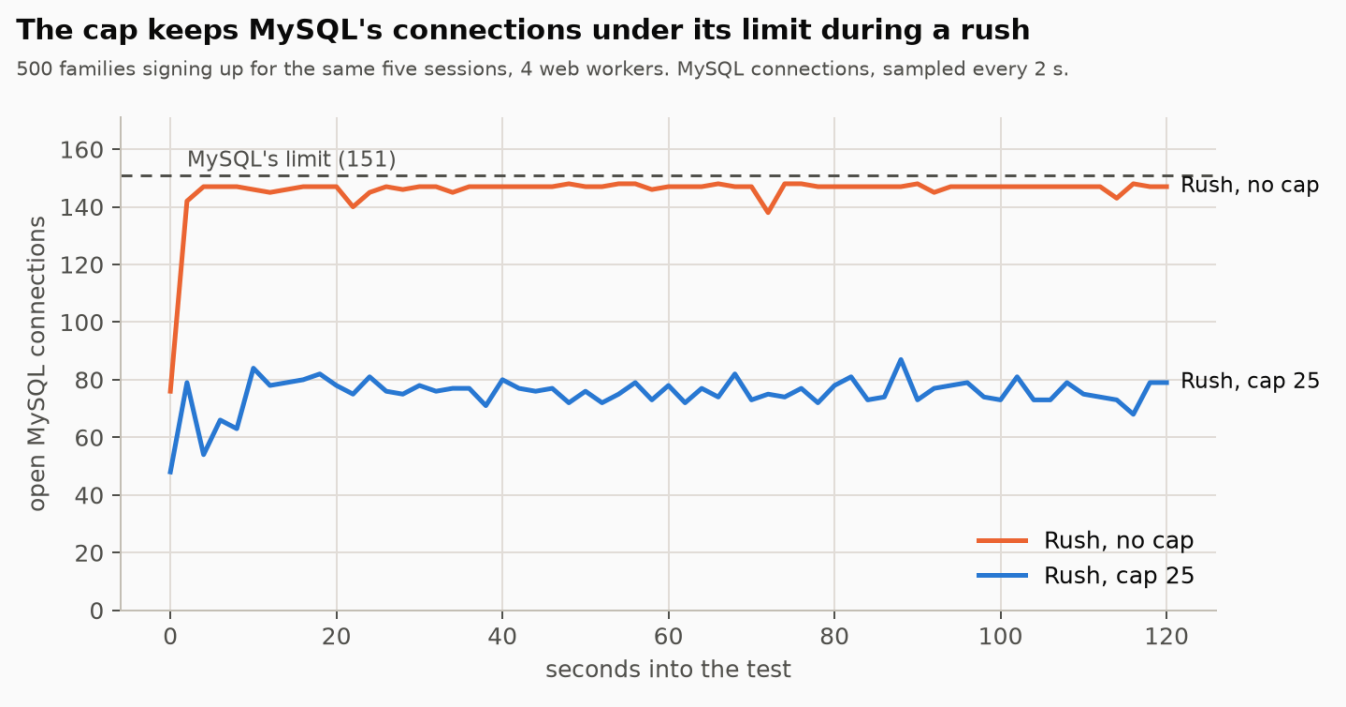
Mixed load; response times in ms.

At the same load (300 users, about 63 requests a second) 4 web workers keep the 95th percentile at 110 ms where 2 let it climb to 610 ms. Logging in takes 250–350 ms: the password hash is slow on purpose.

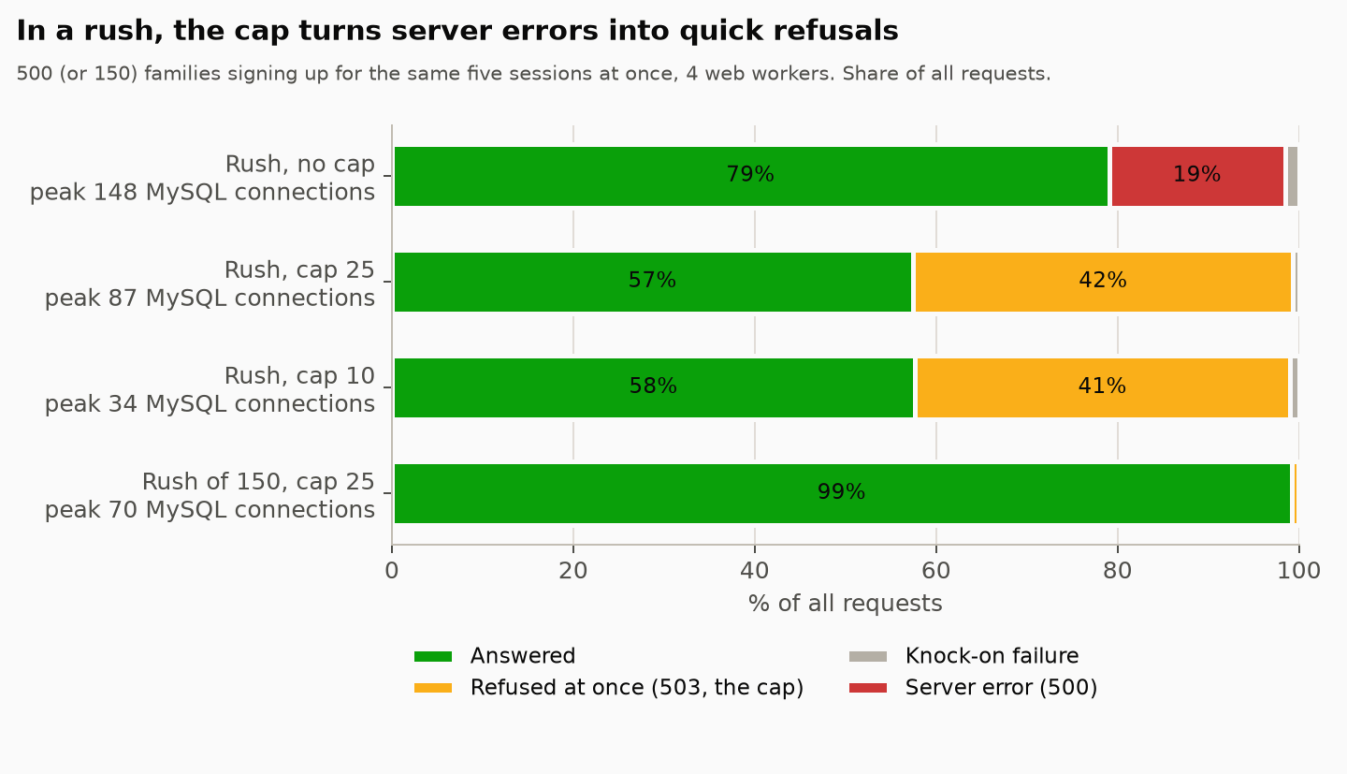
A registration rush

Two problems showed up when 500 families signed up for the same five sessions at once. Both were fixed on 30 September 2026.

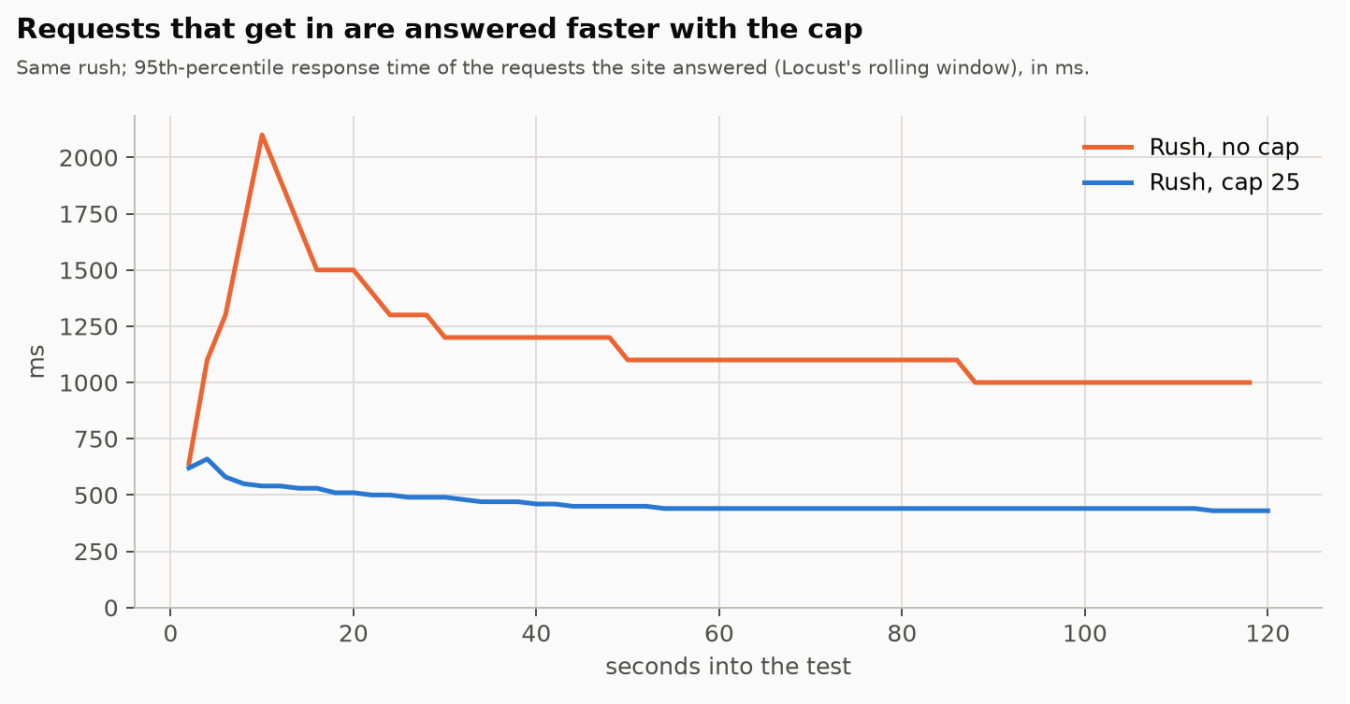
- **Sessions overbooked** (up to 25 confirmed places on a session of 22, with duplicate waiting-list positions): the sign-up read the places taken, then inserted, without a lock. Signing up and cancelling now go through `events.registrations`, which locks the session's row first; real threads in `BookingConcurrencyTests` fail without the lock.
- **MySQL ran out of connections**: under ASGI every request in progress holds its own connection, and uvicorn starts any number of requests at once. `gunicorn.conf.py` now caps each worker at 25 connections (uvicorn's `limit_concurrency`); beyond it the request gets an immediate 503.



500 families, 4 web workers; MySQL connections sampled every 2 s.



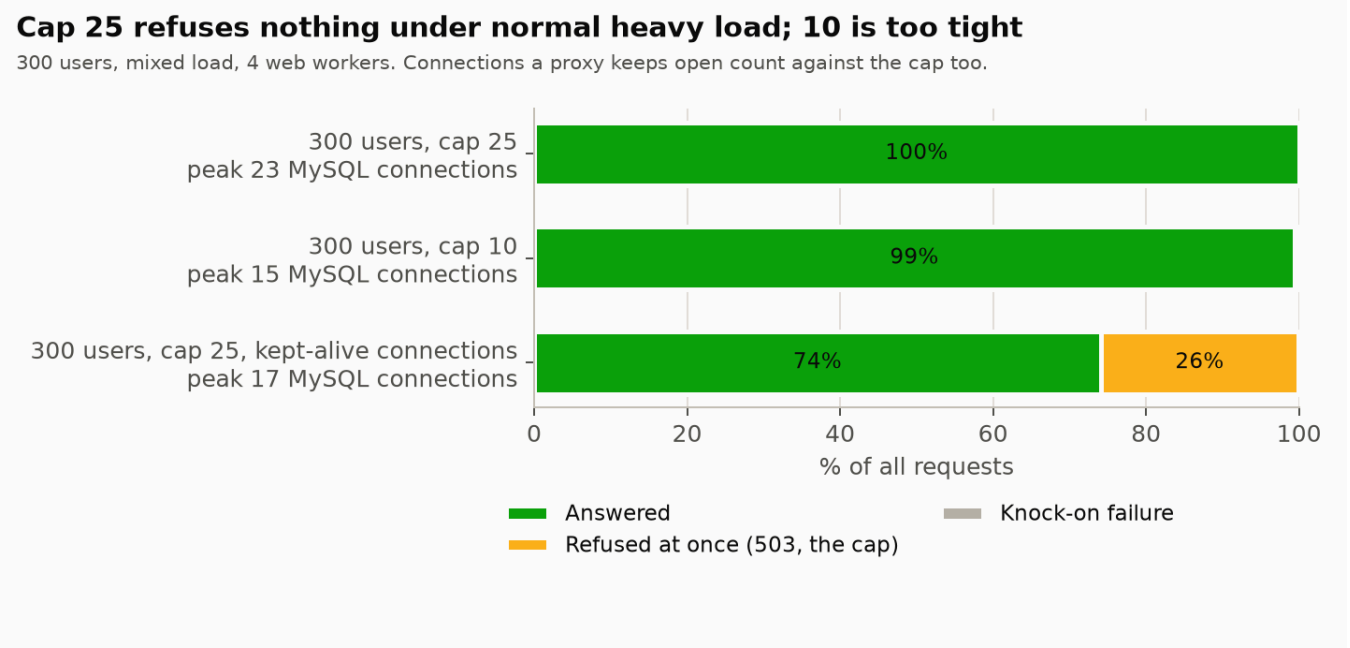
Share of all requests. The same work gets done in each run (about 220 answered requests a second); what changes is how the rest fails.



95th-percentile response time of the answered requests.

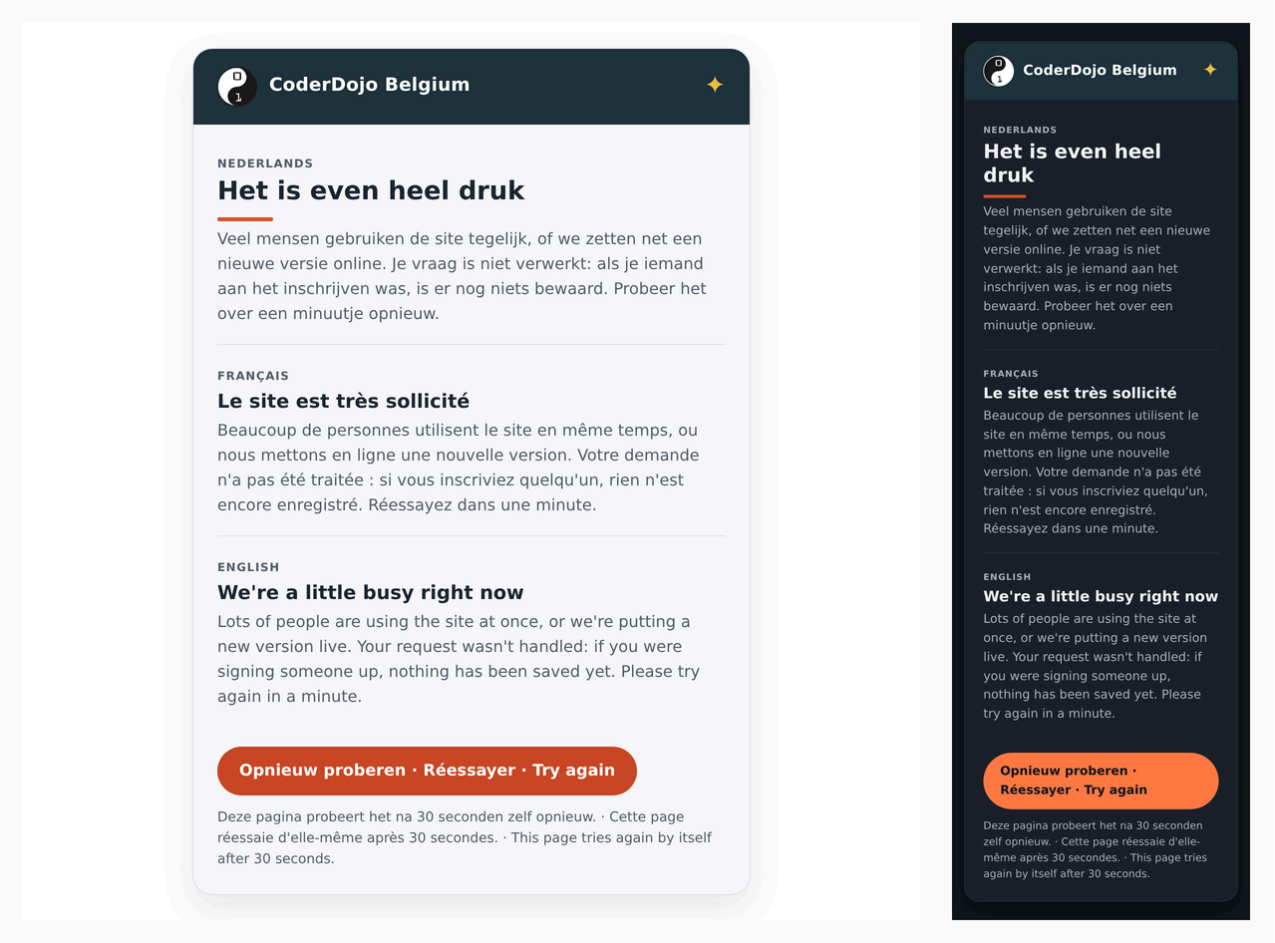
Why. A refused request costs nothing and can be retried; a request that waits for a database connection and then fails holds a worker, a thread and a connection while it waits, which slows everyone down.

Choosing the cap



300 users, mixed load, 4 web workers.

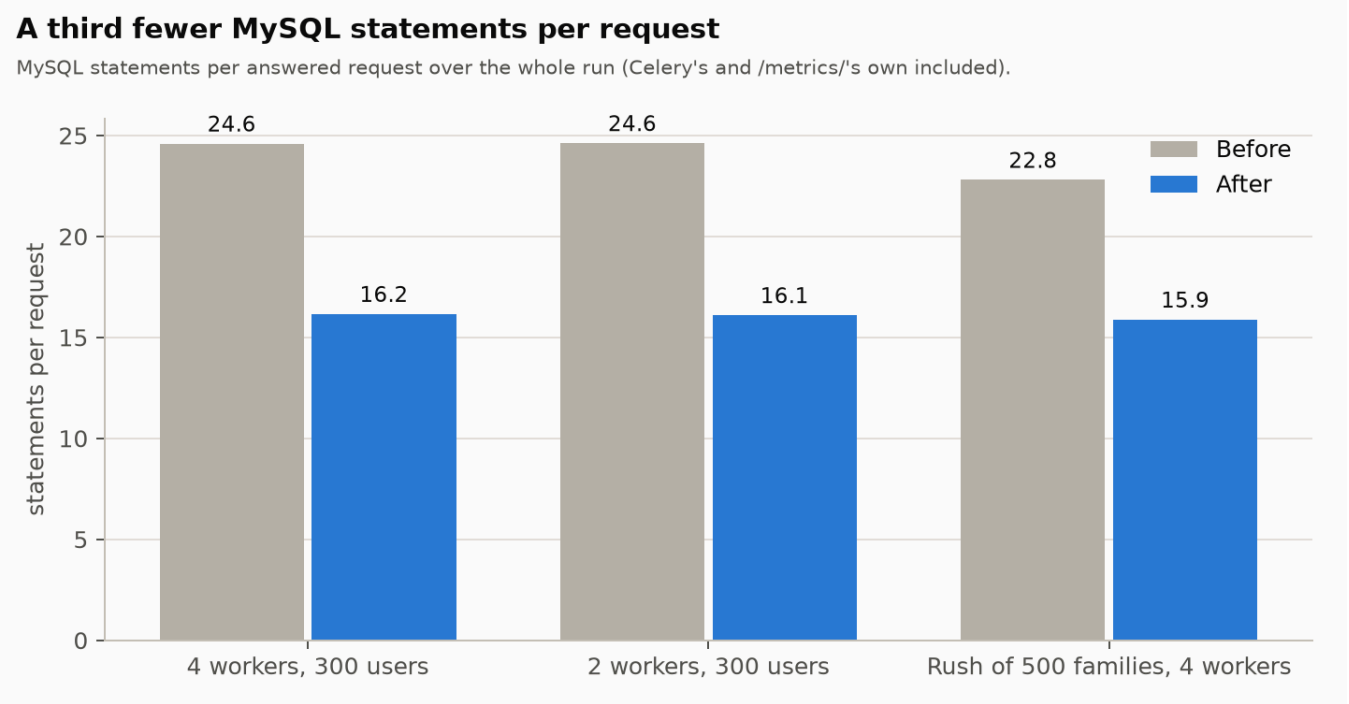
- **25 refuses nothing under normal heavy load** and keeps a rush far from MySQL's limit; 10 already turns normal traffic away. With 4 workers, at most about 100 connections plus Celery's.
- uvicorn counts **open connections**, not requests: connections a proxy keeps open between requests, and open notification WebSockets, take places too. How Level27's proxy connects to gunicorn is still to confirm.
- `scripts/deploy.sh --check` says whether gunicorn reads the file (it doesn't when it starts outside `~/app` or with a `-c` of its own).
- **A refused visitor sees our own page**, not uvicorn's bare "Service Unavailable": the devcontainer's nginx serves `errors/busy.html` (the site's design, Dutch, French and English, no script, trying again after 30 seconds) for a 502, 503 or 504 from the site, keeping the status and adding `Retry-After`; `/health/` and the API keep their own answers. It's the example for Level27's proxy.



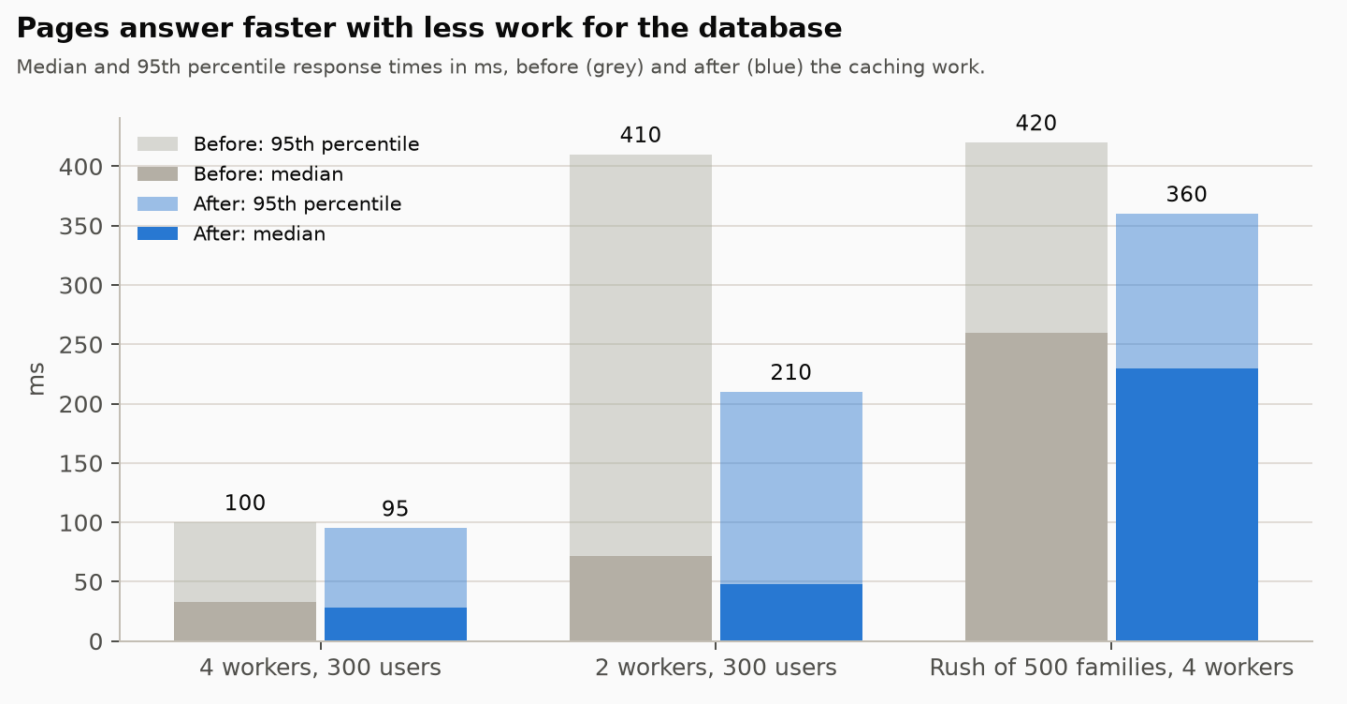
The proxy's busy page (errors/busy.html): desktop in light mode, a phone in dark mode.

Caching: less work for the database

The pages used to redo work the database had already done: a count query per session card, the account's roles and dojos several times per request, the session row and the sign-in policy on every request, and site-wide content on every visit. Since 1 October 2026 the lists count in the same query, sessions are read from Redis (and still written to MySQL), the account's nav and the policy are cached, and so are the site-wide content, the events list's first page and each dojo's page. A public page now makes 1 query instead of 8 to 24.



The same runs on the old and the new code: MySQL statements per answered request.

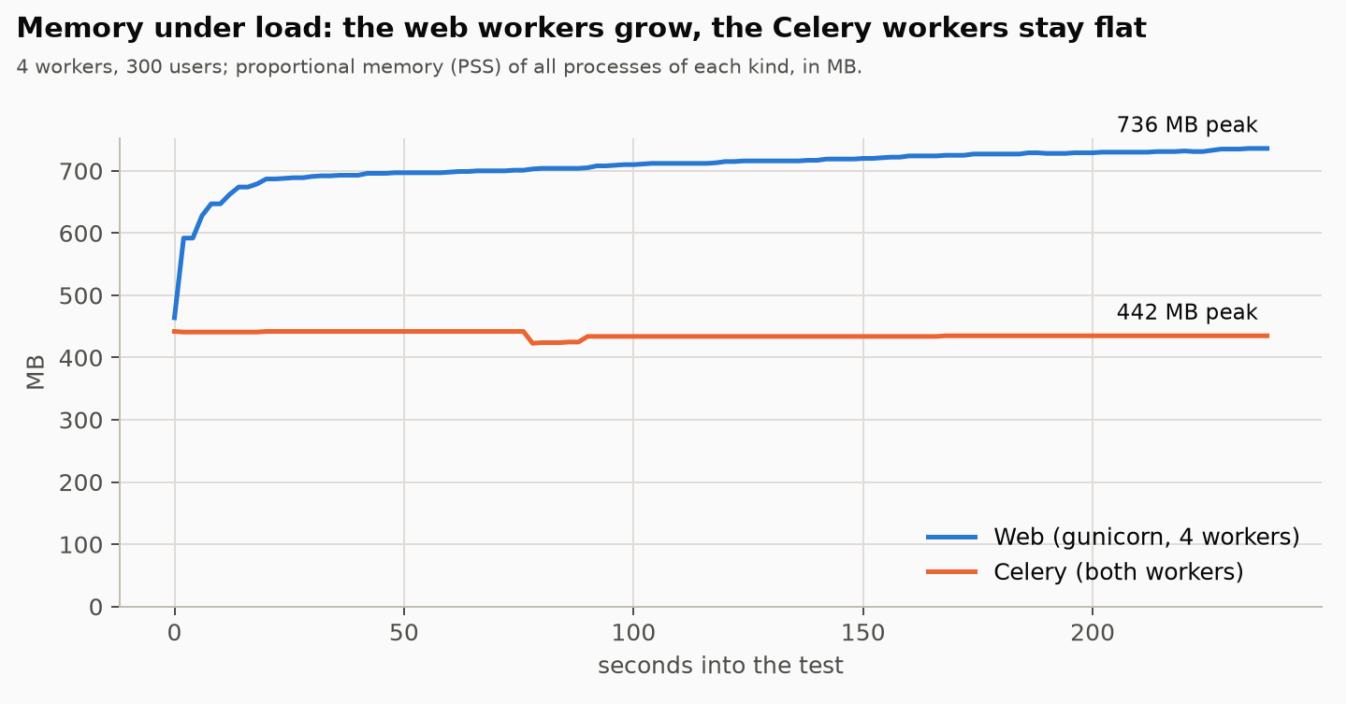


Median and 95th percentile, before (grey) and after (blue).

- **A third fewer statements per request** (24.6 to 16.2). The rest is logged-in families booking, attendance marks (writes) and 2 statements to open each request's own MySQL connection, which no cache saves.
- **Where the site was short of capacity, it shows**: with 2 web workers the 95th percentile halved (410 to 210 ms) and the 0.8% of failed requests went away; in a rush, 17% more requests were answered in the same time.
- Site-wide content is found in the cache 99.9% of the time; the events list's first page about 70%, because every booking clears it so the places left stay right. Redis grew by 0.2 MB.

> **Why.** A cache that's cleared on every save shows a change at once; one that only expires would show a session as open for minutes after it filled up.

Memory

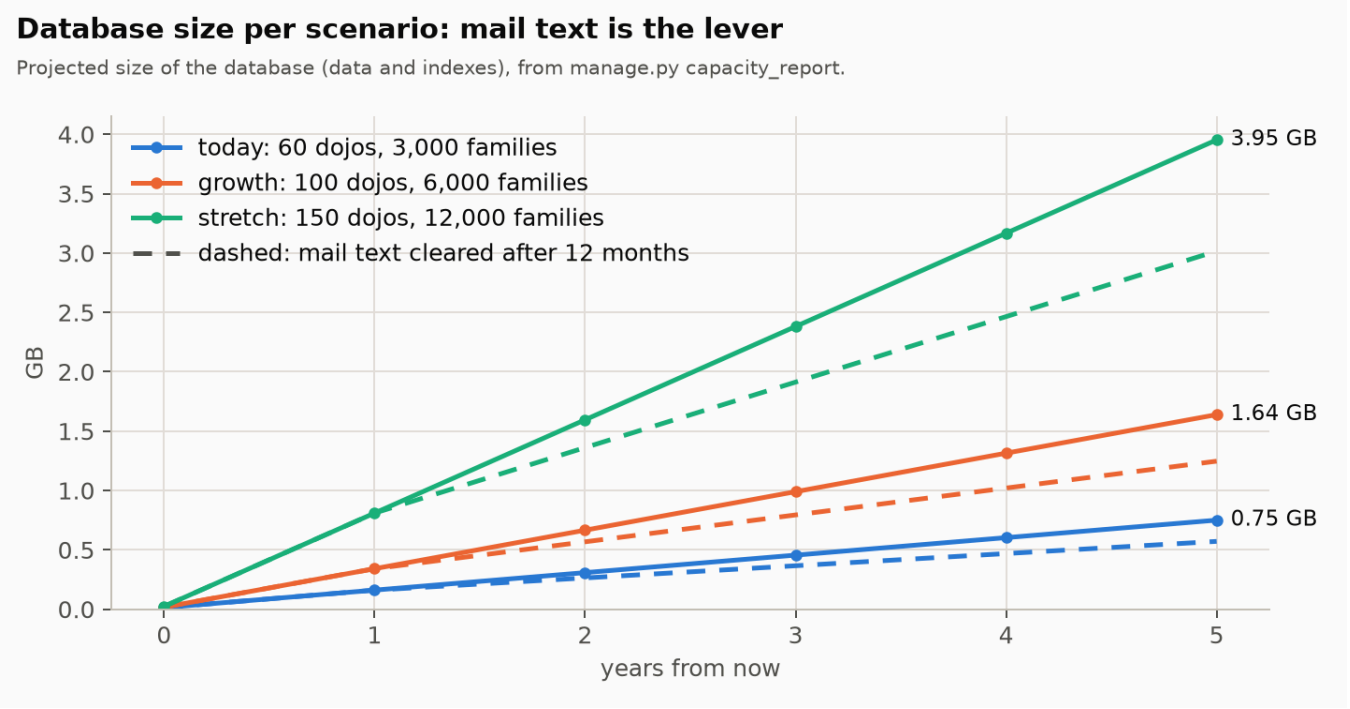


4 web workers, 300 users; PSS of all processes of each kind.

Component	Idle	Peak
Web: gunicorn master and 4 uvicorn workers	460 MB	750 MB (855 in an uncapped rush)
Celery periodic worker (parent, pool child, beat)	255 MB	255 MB
Celery mailing worker (parent, pool child)	160 MB	385 MB, nightly engagement rebuild
Redis (cache, Channels, broker)	3 MB	5 MB
Total	**0.9 GB**	**about 1.45 GB: plan 2 GB**

The mailing worker's child reaches 300 MB during the nightly rebuild and is then replaced (`--max-memory-per-child` , 200 MB); measured before 3 October 2026, when the rebuild's own peak went from 132 to 35 MB (section 16, "Memory per function"), so that peak should now be lower; a campaign launch stays at 160 MB because audiences are queued in chunks. Mail goes out at 120 a minute by design: a campaign to 6,600 families takes about 55 minutes, with booking mail ahead of it. Since 2 October 2026 a booking confirmation sent during a campaign waits about 16 seconds instead of two minutes: the dispatcher keeps only two batches in flight (the broker sends first in, first out, so everything claimed ahead was a wait), and a campaign is queued 200 accounts per task, each queueing the next behind what's waiting.

Database growth



Projected size per scenario; dashed: mail text cleared after 12 months.

- About 350 MB a year in the growth scenario, 1.6 GB after five years. 70% is the mail log: every mail keeps its rendered text. Clearing it after 12 months, as the privacy register already says, saves about a quarter.
- Accounts, children and bookings are anonymised, never deleted, so those tables only grow; the audit log is the second-largest (525 bytes an entry).

Disk and uploads

On disk	Size
Database, binary logs, backups	1.6 GB after five years, plus tens of MB of logs and a copy per backup
Uploaded images (`media/`)	about 70 MB a year, made smaller when saved (0.8 GB as uploaded)
Standard images	a few MB: copied once, shared by every row that uses one
Code releases (5 kept), Python packages	7 MB a release, 236 MB
Celery's logs (journal)	small: the periodic worker logs at WARNING (at INFO it wrote about 6 MB a day)

Teams upload dojo icons and session banners, the organisation promotion images, sponsor logos and badge icons, volunteers their background-check document. Every upload goes through `core/uploads.py`:

- **Limits**: an image at most 10 MB and 40 megapixels, a real raster image (never an SVG); the background-check document at most 10 MB and a PDF, JPEG or PNG by its first bytes. The proxy refuses bodies over 12 MB.
- **Made smaller**: at most 1600 px (banners), 800 px (logos) or 512 px (icons, photos), re-encoded as JPEG, or PNG with transparency.

- **No metadata survives**: the file is rebuilt from its pixels, so EXIF (a phone photo's GPS position), XMP, comments, text chunks and colour profiles are left out, after the orientation and colour profile are applied.
- **Nothing left behind**: a replaced image, or one whose row is deleted, is deleted on commit, unless it's a standard image or another row uses it.

> **Why.** Uploaded photos are public files: without this a banner kept the phone's GPS position of where it was taken, and a single photo could weigh more than a year of bookings.

Running it again, on production too

- Read-only and safe on production: `manage.py capacity_report`, `/metrics/` with its token, the daily `CapacitySample`, and `deploy.sh --check`.
- A public load test (`loadtest/run.sh ... public`: visitors only, nothing written) at a quiet moment, starting at 50 users; it goes through Level27's proxy, so it's also the end-to-end check.
- The modes that log in and book refuse any host that isn't local: they'd book real sessions and mail real families.

16 Coding standards and quality

How code is written, checked and measured: the standards, the linting setup, the tests and what they cover, how complex the code is and how much memory each function uses. Measured on 2 and 3 October 2026; the full versions, with how to measure again, are `CODING_STANDARDS.md` and `MEMORY_PROFILE.md`, and the conventions themselves are in `CLAUDE.md`.

Production code and development-only code

Ships to production	Stays with developers
The apps and `website/`, `locale/`, `main.py`, `manage.py`, `requirements.txt`, `gunicorn.conf.py`, the Celery systemd units	`.devcontainer/`, `requirements-dev.txt`, `pyproject.toml`, `loadtest/`, `quality/`, `user-journeys/`, `scripts/deploy.sh`, `.github/` (CI), `docs/` (published to GitHub Pages)
In between: tests, seeders and import commands sit inside the apps and ship, but never run there (the import commands' libraries aren't installed; `totp_code` and `simulate_bounce` need `DEBUG`); the debug toolbar and django-silk only exist while `DEBUG` is on.	

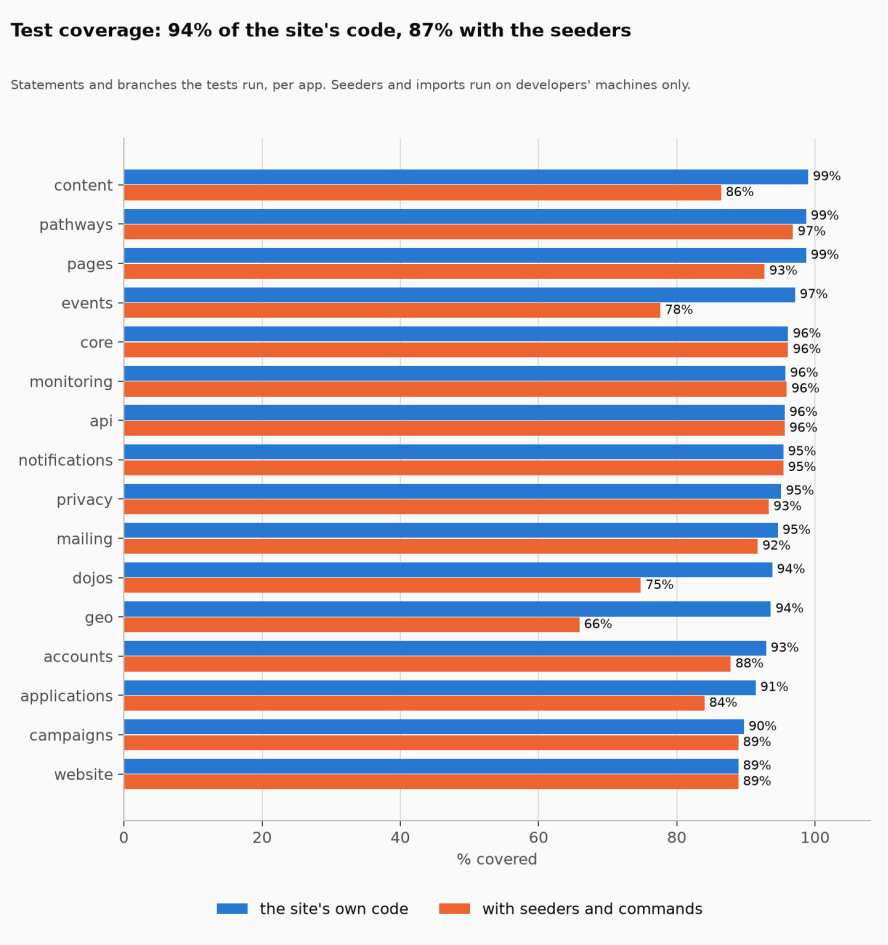
Standards and linting

- **Ruff**, pinned and configured in `pyproject.toml`: pycodestyle, pyflakes, import order, bugbear, flake8-django and pyupgrade; the formatter decides the style (119 characters, double quotes). Security rules (bandit) run separately; a reviewed line gets `noqa` with its reason.
- The devcontainer's editor lints as you type and formats on save with the same Ruff.
- CI: Ruff, the security rules, `pip-audit`, `manage.py check --deploy`, `mypy` and CodeQL in **Code audit**; missing migrations and the whole suite in **Tests**.
- Gated since 2 October 2026: no function over complexity 20 (Ruff `C901`, seed code excepted) and the layers between the apps (`lint-imports`). Not gated, on purpose: a minimum coverage. It's measured.
- **Type checking, gradually** (since 6 October 2026): mypy with django-stubs' Django plugin checks the service modules every view calls (seven to start: onboarding, the team rules, awards, sign-ups, the mail engine and its queue actions, retention), fully annotated; views, forms and tests stay unannotated. Its first run found a timestamp compared with a plain date in the retention job and a milestone badge that could make marking attendance fail.
- **Accessibility: WCAG 2.2 AA** (since 6 October 2026): a skip link and one main landmark on every page, headings that never skip a level, form errors linked to their fields, contrast through the design tokens in both themes, 24px targets, and other-language text marked with its language. Guard tests run in CI; an axe-core scan of every journey page, in both themes and at 320px wide, runs from the host (`check_a11y.py`).

Tests

1,191 tests (13,700 lines of test code for 26,200 lines of code, 6 October 2026). Routes are tested for status, template, gating and the database effect of a POST; services with their refusals; concurrency with real threads. About twenty **guard tests** fail on new code that forgets a rule: privacy classification, export coverage, an audit-log decision, a usable admin, `__str__` without queries, translated form texts, security headers and inline handlers, vendored scripts and fonts, the API schema. Every run shows a summary on GitHub and marks failures on the failing line.

Test coverage



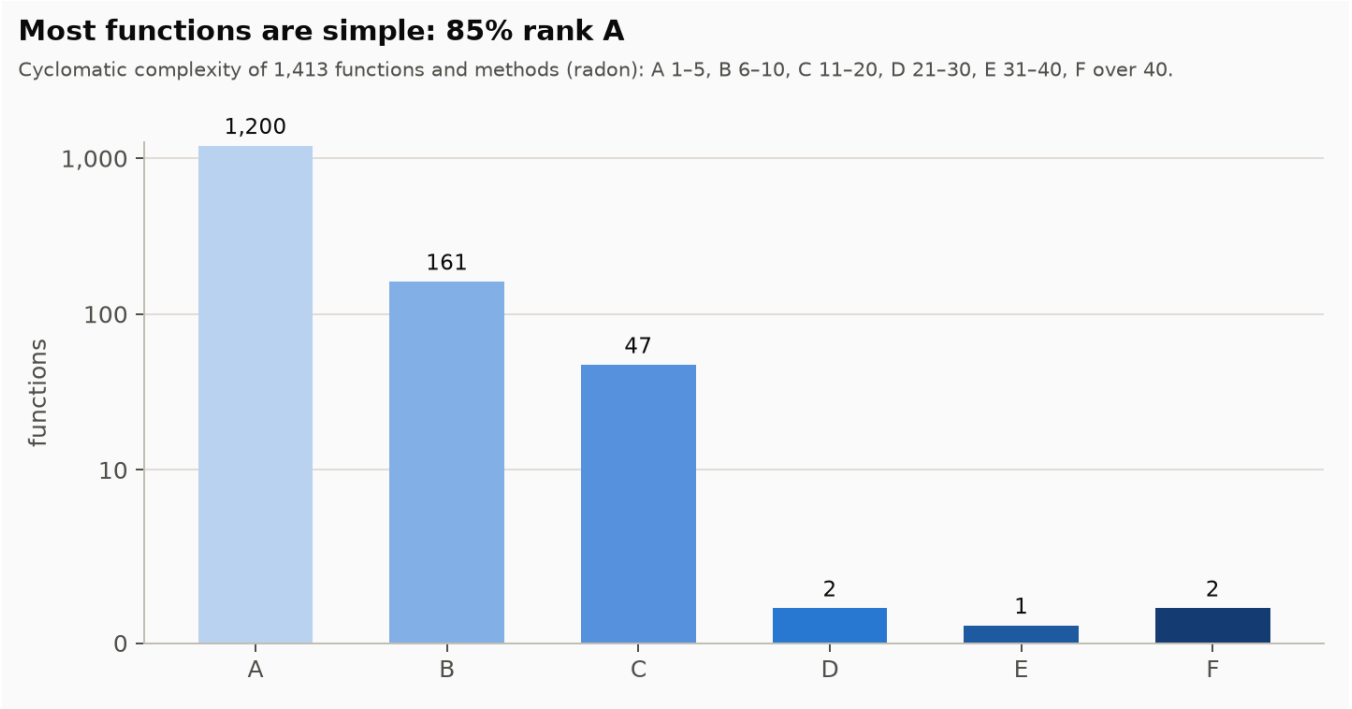
Statements and branches the tests run (coverage.py, branch coverage).

Covered well	Not covered
The business rules and their services (registrations, teams, awards, onboarding, mail, privacy, sign-in), every page's gating, the API, monitoring, every guard rule: the site's own code at 94%, most apps at 91–99%.	Seeders and import commands (51%, development only); refusals of some rules (`dojos/team.py` , `accounts/invitations.py`); two segment attributes; admin actions; outside-service failures; the ASGI routing glue; one-line task wrappers.

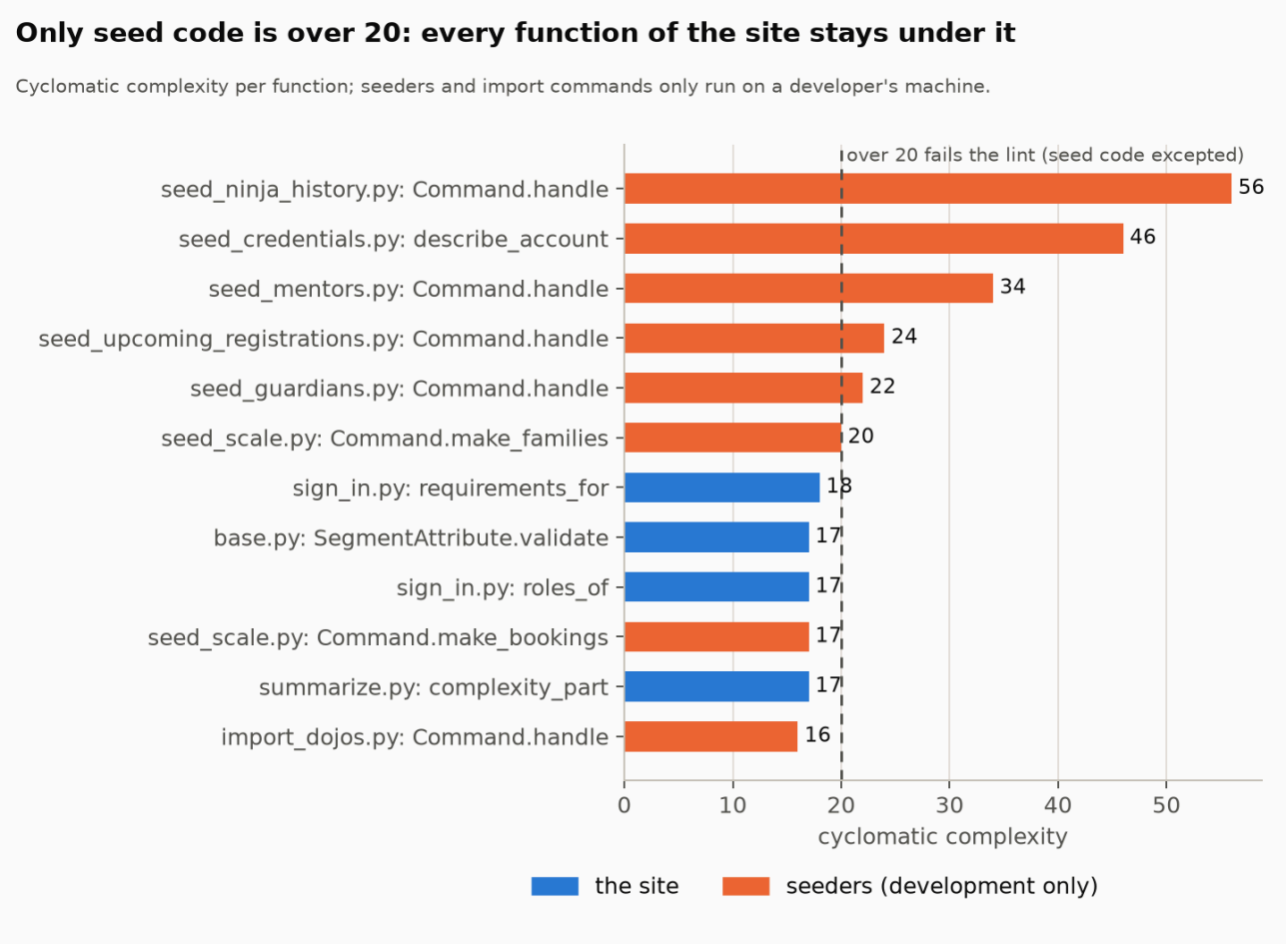
> **Why.** Coverage is a map, not a target: it shows what no test runs, not whether a test checks the right thing. So it isn't gated in CI; the refusals of business rules are the gaps worth closing first.

Complexity

An average complexity of 3.2; every module ranks A for maintainability. The site's six functions over 20 were split on 2 October 2026 (the privacy registry's validation, the engagement rebuild and its metrics, the erasure's collection, `event_signup` , `dojo_team_action`): only seed code is over 20 now, and Ruff fails any new function that is. The three large views modules became packages by area.



Cyclomatic complexity of every function (radon).

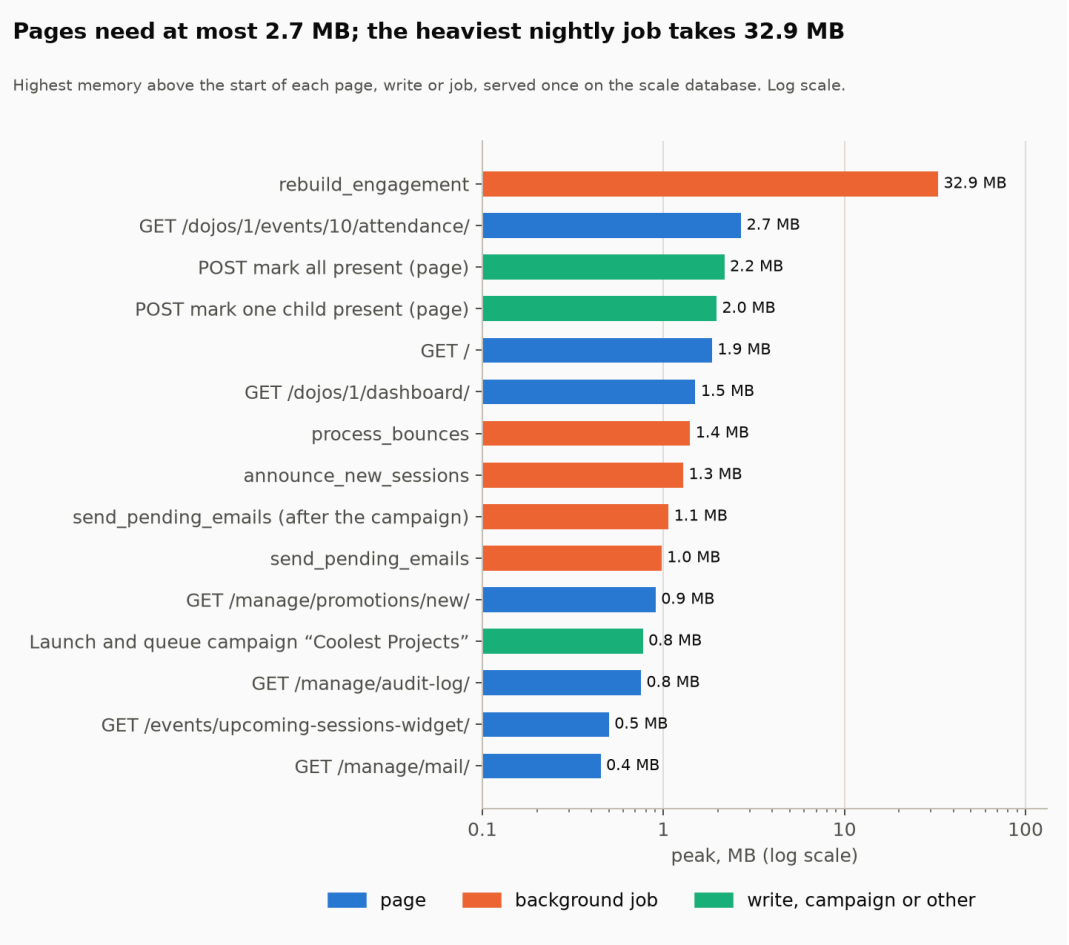


The most complex functions; over 20 fails the lint, seed code excepted.

Memory per function

Measured on 3 October 2026 by `quality/memory_profile.py`: every page (as the account that would open it) and every background job, run once on a `seed_scale` database of a year's growth (8,962 children, 29,400 bookings, 188,926 mails) with production's settings. Python's `tracemalloc` counts the allocations and `sys.monitoring` reports when each of the site's own functions starts and returns, so every function gets its **peak** (the most memory a call needs, with what it calls), its **own peak** (without the site's functions it calls: where to change it) and what it **keeps** after returning. Only Python's allocations count, so processes are bigger (section 15); the figures show where memory goes and how it grows.

Peak per call	Functions
up to 100 KB	643 of 747 (86%)
100 KB to 1 MB	82
1 to 10 MB	19
over 10 MB	3, all the nightly engagement rebuild



Peak memory of the pages, writes and jobs that need the most (log scale).

Function	Peak	Verdict
`events.engagement._registrations_by_ninja`	128 MB, now 10.8 MB	**Refactored on 3 October 2026.** Each of the 29,400 bookings got its own copy of its session and dojo (`select_related` over the whole table), so it grew with every booking. It now loads every session and dojo once and reads the bookings as plain values, and the children's home dojos share the same dojos: the whole nightly rebuild went from 132 MB to 35 MB and from 12.6 to 7.4 seconds, writing the same 22,792 rows.
The attendance list's award forms (`dojos/forms.py`)	30 KB per form, now none	**Fixed on 3 October 2026.** A `lazy()` call per form built a new class each time, kept until the garbage collector ran: 1.6 MB per page for 22 children, now 0.25 MB. `lazy()` is now called once at module level (four other forms too), and a guard test fails on a `lazy()` call inside a function.
Pages	93 KB median, 2.7 MB at most	Fine: bounded by one session or one page of rows.
Mail jobs, campaign queuing, segments, export, retention	0.2–1.4 MB	Fine: they work in chunks or let the database count, whatever the number of mails or accounts.

> **Why.** A job that reads a whole table should read values and share the small tables it refers to, never select_related across it: memory then grows with the small table, not the large one. Found on the way: the Mail queue page spent 6.6 seconds on one count over the mail log; an index on (status, sent_at) brought that count to a millisecond and the page to 0.3 seconds.